Computer Science
12. Database

12주차: 데이터베이스

목차 INDEX

01 데이터베이스

- Database
- DBMS
- RDB
- Table
- Schema
- PK / FK
- 정규화
- Cardinality

02 SQL

- SQL Basics
- SELECT
- WHERE
- ORDER BY
- INSERT / UPDATE
- DELETE
- GROUP BY
- JOIN

03 Index & Search

- Sequential Search
- DB Index
- B-Tree / B+Tree

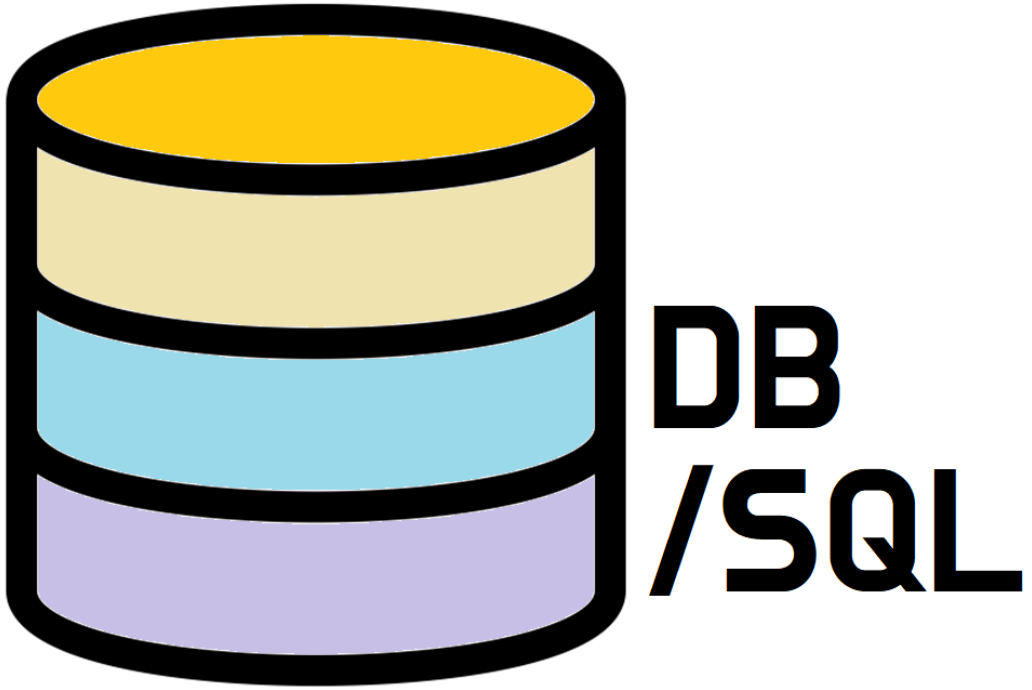
04 Transaction

- Transaction
- Commit
- Rollback
- ACID
- NoSQL
- Redis
- 다양한 DB

01
데이터베이스

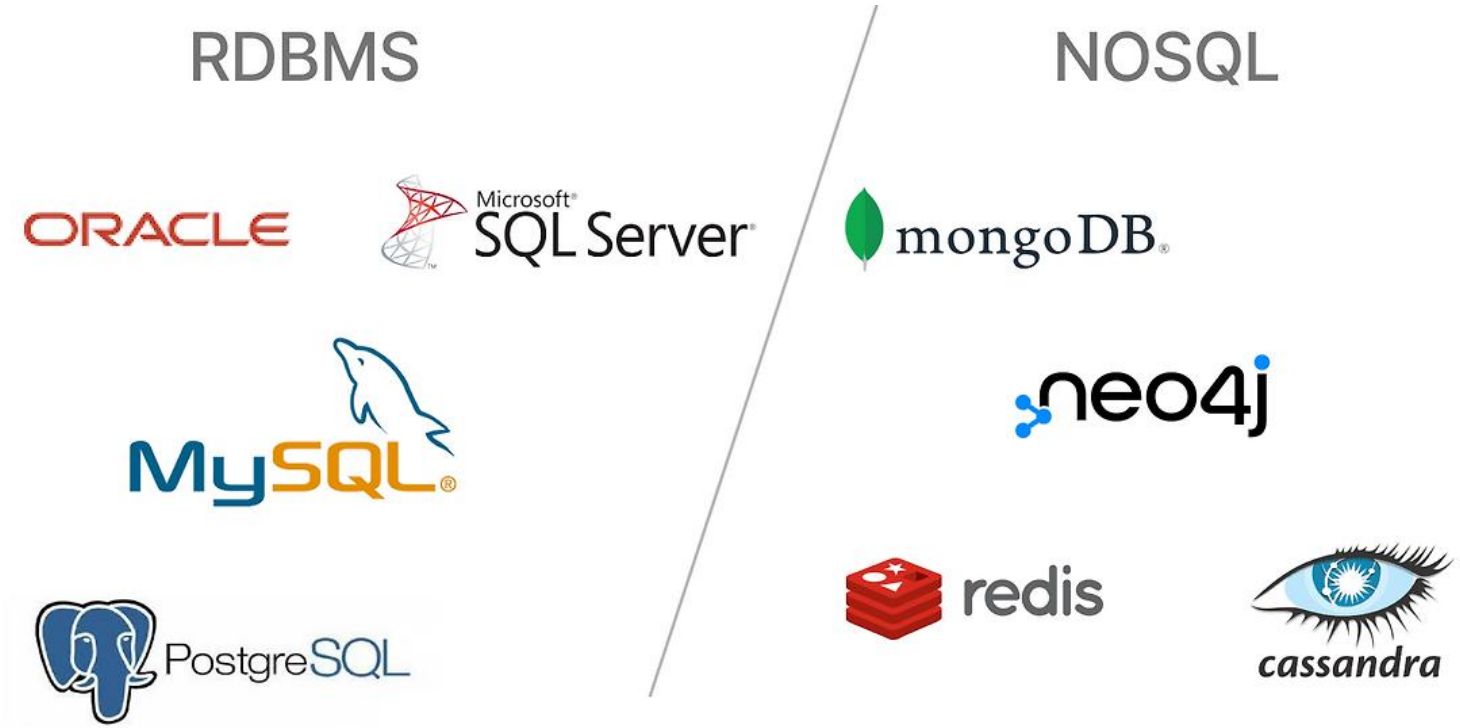
Database

Database



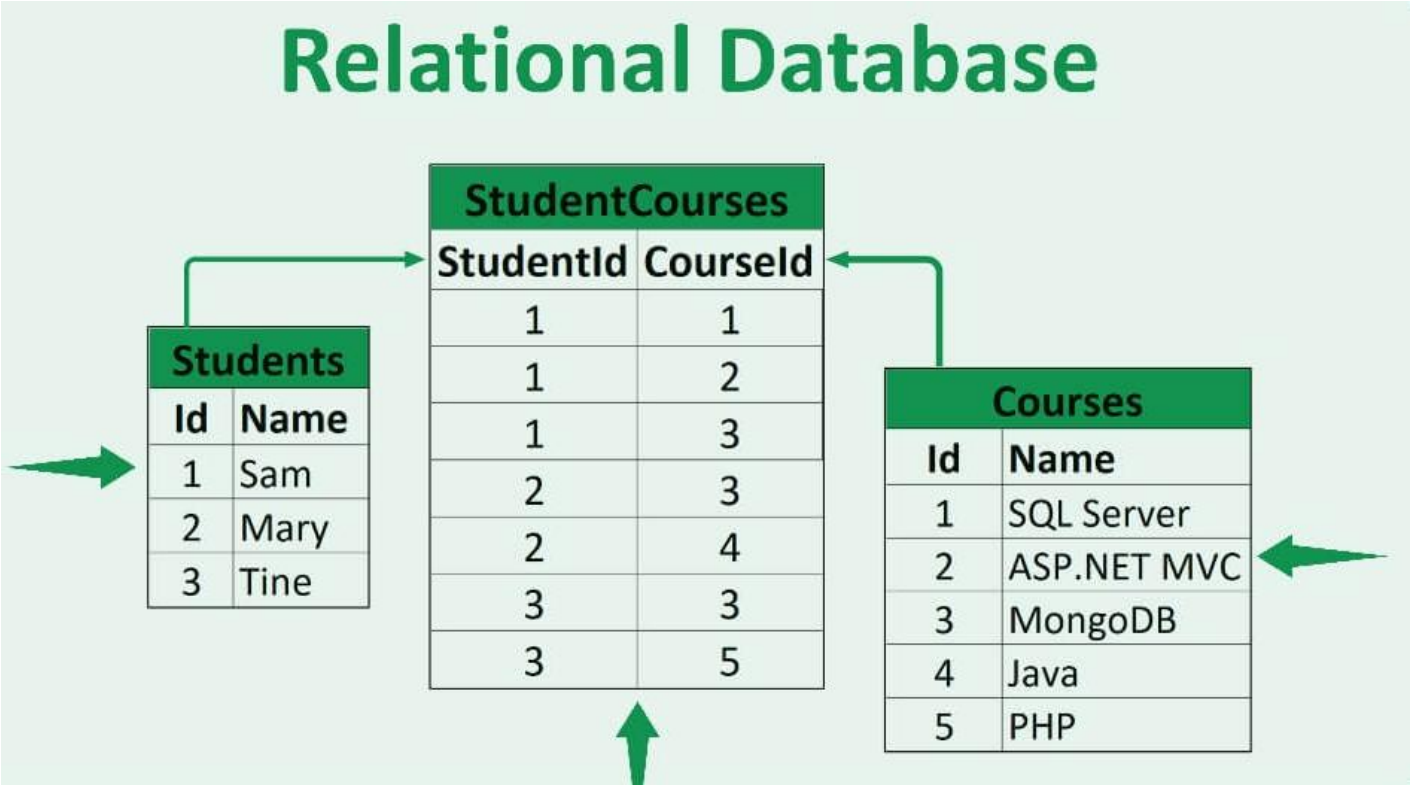
데이터를 체계적으로 저장하고 관리하여 필요한 정보를
효율적으로 검색하고 활용할 수 있도록 구성한 데이터의 집합

DBMS: Database Management System



데이터베이스의 저장, 조회, 수정, 삭제를 비롯해
데이터의 무결성, 동시성 및 복구를 관리하는 소프트웨어

RDB: Relational DB (관계형 데이터베이스)



데이터를 행과 열로 구성된 테이블 형태로 저장하고,
테이블 간의 관계를 통해 데이터를 체계적으로 관리하는 데이터베이스

Schema

Column	Data Type	Constraint
product_id	INTEGER	PRIMARY KEY
name	VARCHAR(100)	NOT NULL
price	INTEGER	NOT NULL
stock	INTEGER	DEFAULT 0

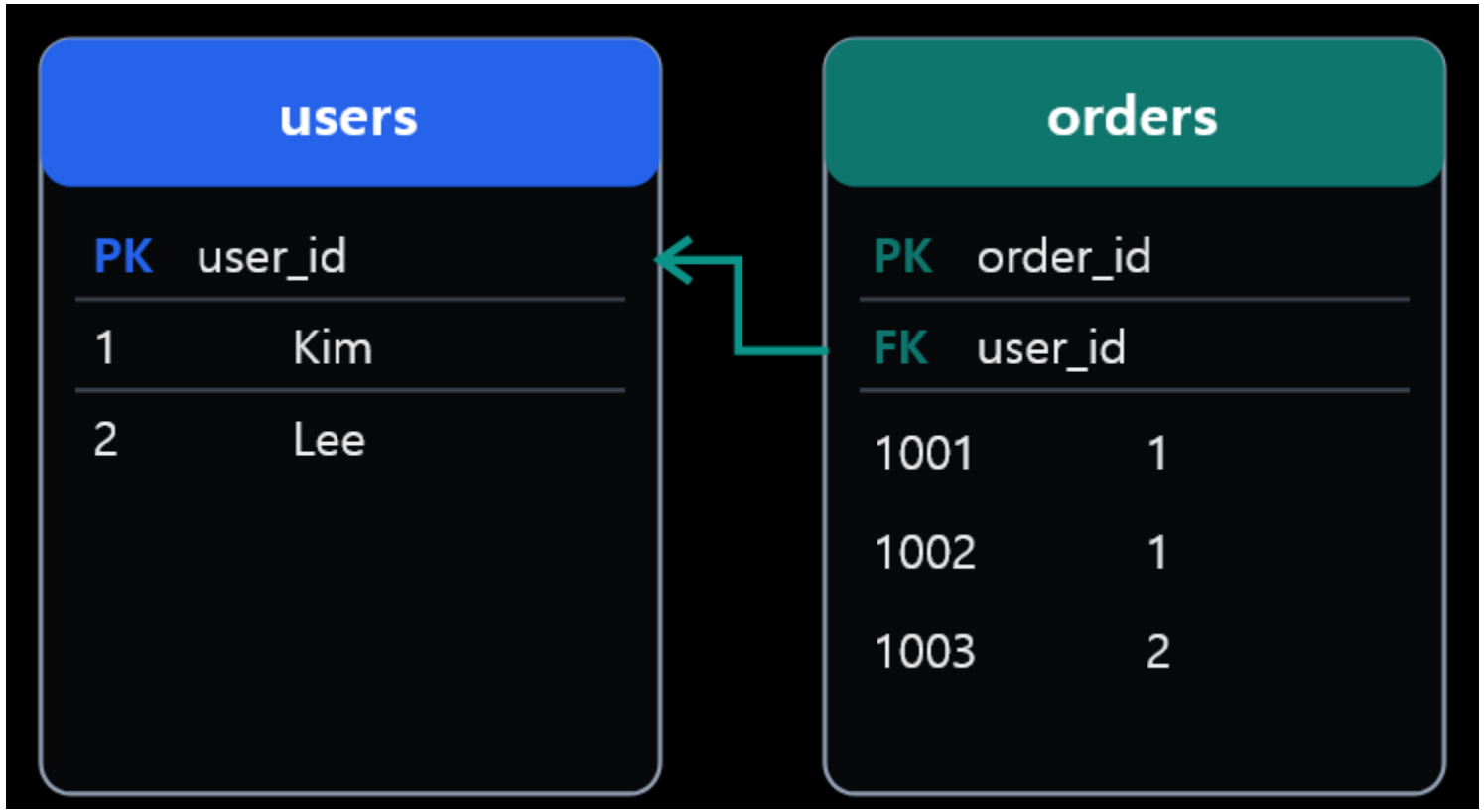
데이터베이스에서 데이터가 어떤 구조로 저장되는지 정의한 설계도
테이블 구성, 컬럼의 이름 및 자료형, 조건 및 테이블 간 관계 명시

Constraint

제약 조건	설명
PRIMARY KEY	각 행을 고유하게 식별하며 중복 및 NULL 금지
FOREIGN KEY	다른 테이블의 키를 참조하여 참조 무결성 유지
UNIQUE	컬럼 값의 중복 방지
NOT NULL	NULL 값 저장 금지
CHECK	지정된 조건을 만족하는 값만 허용
DEFAULT	값을 지정하지 않았을 때 기본값 설정

데이터베이스에 잘못된 데이터가 저장되지 않도록
테이블이나 컬럼에 설정하는 규칙
스키마 안에 포함되며, 데이터의 조건을 정의함

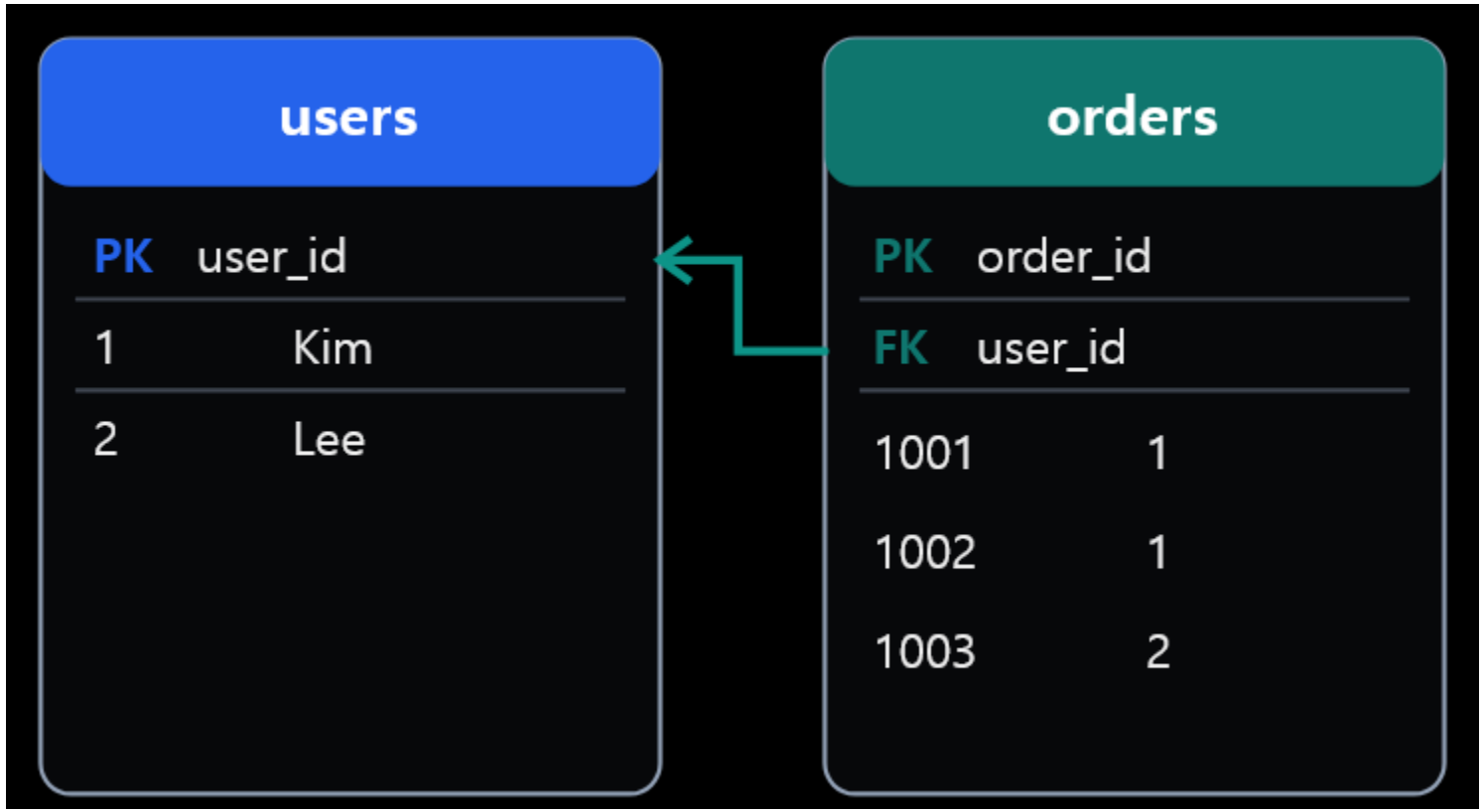
Primary Key & Foreign Key



Primary Key, 기본 키 (PK)

테이블에서 각 행을 식별하기 위한 고유한 키

Primary Key & Foreign Key



Foreign Key, 외래 키 (FK)

다른 테이블의 기본 키 또는 UNIQUE를 참조해 테이블 간 관계를 정의하는 키

정규화 (Normalization)

Before Normalization

Employee_Department

Emp_ID	Emp_Name	Department	Dept_Location	Emp_Skills
101	Nick Wise	HR	London	Recruitment, Payroll
102	John Cader	Finance	Australia	Budgeting
103	Lily Case	HR	London	Recruitment
104	Ford Dawid	IT	Chicago	Programming, Testing



After Normalization

Employee

Emp_ID	Emp_Name	Dept_ID
101	Nick Wise	D1
102	John Cader	D2
103	Lily Case	D1
104	Ford Dawid	D3

Department

Dept_ID	Department	Dept_Location
D1	HR	London
D2	Finance	Australia
D3	IT	Chicago

Employee Skills

Emp_ID	Emp_Skills
101	Recruitment
101	Payroll
102	Budgeting
103	Recruitment
104	Programming
104	Testing

정규화

데이터 중복과 기타 이상현상을 최소화하기 위한 테이블 설계 방법론
정규화를 만족하는 DB는 CRUD가 쉬워짐

제 1 정규화 (1NF, First Normal Form)

- 데이터베이스에의 한 행에는 하나의 값만 들어가야 함
- 제 1 정규화를 만족하는 테이블을 제 1 정규형 테이블이라 부름



제 2 정규화 (2NF, Second Normal Form)

- 여러 컬럼으로 이루어지는 PK를 Composite Primary Key라 함
- 이 CPK의 일부에만 종속되는 컬럼은 별도의 테이블로 분리해야 함
- 제 1 정규형 만족



제 3 정규화 (3NF, Third Normal Form)

- 그런데 기본 키 말고 다른
  일반 컬럼에 종속되는 경우도 있음
- 종속 관계가 기본 키 > 일반 컬럼 >
  종속되는 컬럼일 경우,
  이 컬럼을 별도의 테이블로
  분리해야 함
- 제 2 정규형 만족

정규화 이전 — 2NF 위반

주문상품 테이블 — 기본 키: (주문 ID, 상품 ID)

주문 ID (PK)	상품 ID (PK)	상품명	수량
1001	P01	키보드	2
1001	P02	마우스	1
1002	P01	키보드	1

↓

부분 종속된 상품 정보를 별도 테이블로 분리

제2정규화 이후 — 2NF 충족

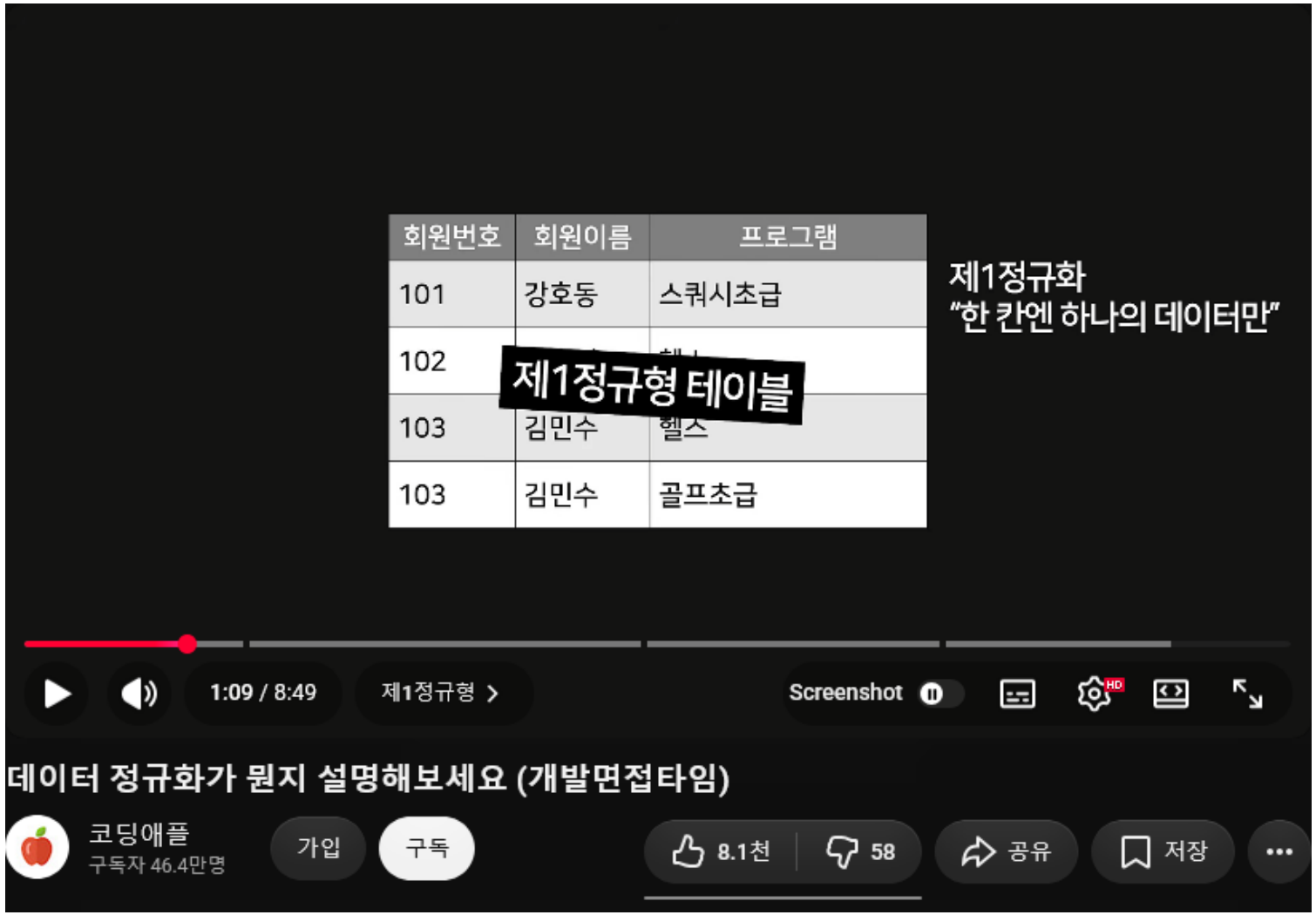
① 주문상품 (Order Items)

주문 ID (PK)	상품 ID (PK, FK)	수량
1001	P01	2
1001	P02	1
1002	P01	1

② 상품 (Products)

상품 ID (PK)	상품명
P01	키보드
P02	마우스

Normal Form


회원번호 회원이름 프로그램
101 강호동 스쿼시초급
102
103 김민수
103 김민수 골프초급
제1정규형 테이블
제1정규화
"한 칸엔 하나의 데이터만"
1:09 / 8:49
제1정규형 >
Screenshot
데이터 정규화가 뭔지 설명해보세요 (개발면접타임)
코딩애플
구독자 46.4만명
가입
구독
8.1천
58
공유
저장

슈퍼 키 (여기부터 심화내용)

학번	이름	이메일
1001	김철수	kim@abc.com
1002	이영희	lee@abc.com
1003	박민수	park@abc.com

슈퍼 키 (Super Key)

테이블에서 각 행을 유일하게 식별할 수 있는 하나 이상의 컬럼 조합
위의 테이블에서는 학번, 이메일, 학번+이름, 학번+이메일,
이름+이메일, 학번+이름+이메일이 슈퍼 키에 해당함

후보 키

학번	이름	이메일
1001	김철수	kim@abc.com
1002	이영희	lee@abc.com
1003	박민수	park@abc.com

후보 키 (Candidate Key)

각 행을 유일하게 식별할 수 있어야 하면서,
유일성을 만족하는 데 불필요한 컬럼이 없는 조합
위의 테이블에서는 학번, 이메일이 후보 키에 해당함

기본 키

학번	이름	이메일
1001	김철수	kim@abc.com
1002	이영희	lee@abc.com
1003	박민수	park@abc.com

기본 키 (Primary Key)

후보 키의 컬럼 조합 중 하나를 선택한 것

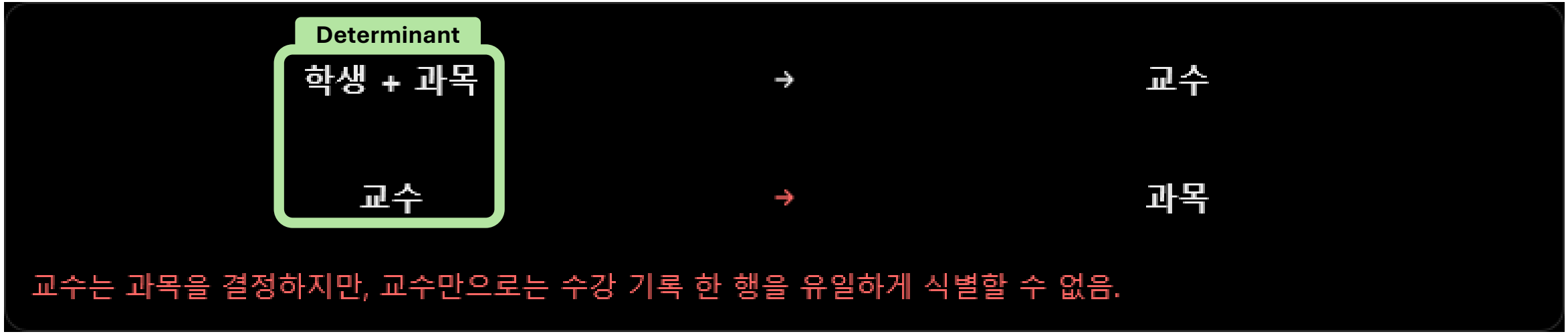
만약 조합에 컬럼이 2개 이상이었으면 Composite Primary Key(복합 기본 키)라고 부름

보이스-코드 정규화 (BCNF, Boyce-Codd Normal Form)

학생	과목	교수
철수	DB	김교수
영희	DB	이교수
철수	OS	박교수

- 한 학생은 같은 과목을 한 교수에게서만 수강
  - 교수는 하나의 과목만 담당
  - 한 과목은 여러 교수가 담당 가능

보이스-코드 정규화 (BCNF, Boyce-Codd Normal Form)

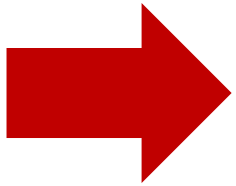


- 종속은 위와 같음
- 제 3 정규형은 만족하는 상태
- 다른 컬럼의 값을 결정하는 컬럼(의 조합)을 Determinant라 부름

보이스-코드 정규화 (BCNF, Boyce-Codd Normal Form)

- 모든 결정자가 슈퍼 키가 되도록 테이블을 구성
- 제 3 정규형을 만족함

학생	과목	교수
철수	DB	김교수
영희	DB	이교수
철수	OS	박교수



정규화 이후 — BCNF 충족

① 교수 담당 과목

교수 (PK)	과목
김교수	DB
이교수	DB
박교수	OS

② 학생 수강

학생 (PK)	교수 (PK, FK)
철수	김교수
영희	이교수
철수	박교수

✓ 교수와 과목의 종속 관계를 별도 테이블로 분리하여 BCNF를 만족함.

제 4 정규화 (4NF, Fourth Normal Form)

정규화 이전

학생	언어	취미
철수	영어	축구
철수	영어	독서
철수	일본어	축구
철수	일본어	독서

- 학생이 여러 언어를 구사하고, 여러 취미를 가지고 있는 상황
  - 종속 관계는 학생>언어, 학생>취미로 서로 독립적임
- 이런 종속관계를 Multivalued Dependency, 다치 종속이라 부름

제 4 정규화 (4NF, Fourth Normal Form)

제4정규화 이후

4NF 충족

① 학생 언어

학생	언어
철수	영어
철수	일본어

② 학생 취미

학생	취미
철수	축구
철수	독서

✓ 언어와 취미를 독립적으로 저장하므로 불필요한 조합과 중복을 제거할 수 있음.

- 서로 독립적인 다치 종속 관계를 별도의 테이블로 분리
- 보이스-코드 정규형 만족

제 5 정규화 (5NF, Fifth Normal Form)

정규화 이전 5NF 위반 사례

공급업체 · 부품 · 프로젝트 테이블

공급업체	부품	프로젝트
A	나사	P1
A	볼트	P1
A	나사	P2
B	나사	P1

- 공급업체가 해당 부품을 취급함
- 공급업체가 해당 프로젝트에 참여함
- 프로젝트에서 해당 부품을 사용함

위 조건들이 모두 성립할 때 납품가능

제 5 정규화 (5NF, Fifth Normal Form)

① 공급업체–부품

업체	부품
A	나사
A	볼트
B	나사

② 공급업체–프로젝트

업체	프로젝트
A	P1
A	P2
B	P1

③ 프로젝트–부품

프로젝트	부품
P1	나사
P1	볼트
P2	나사

제 5 정규화 (5NF, Fifth Normal Form)

- 테이블을 여러 개로 분리했다가 합쳤을 때 (SQL JOIN)원래 데이터를 정확하게 복원할 수 있는 관계는 별도 테이블로 분리
- 제 4 정규형 만족

① 공급업체–부품

업체	부품
A	나사
A	볼트
B	나사

② 공급업체–프로젝트

업체	프로젝트
A	P1
A	P2
B	P1

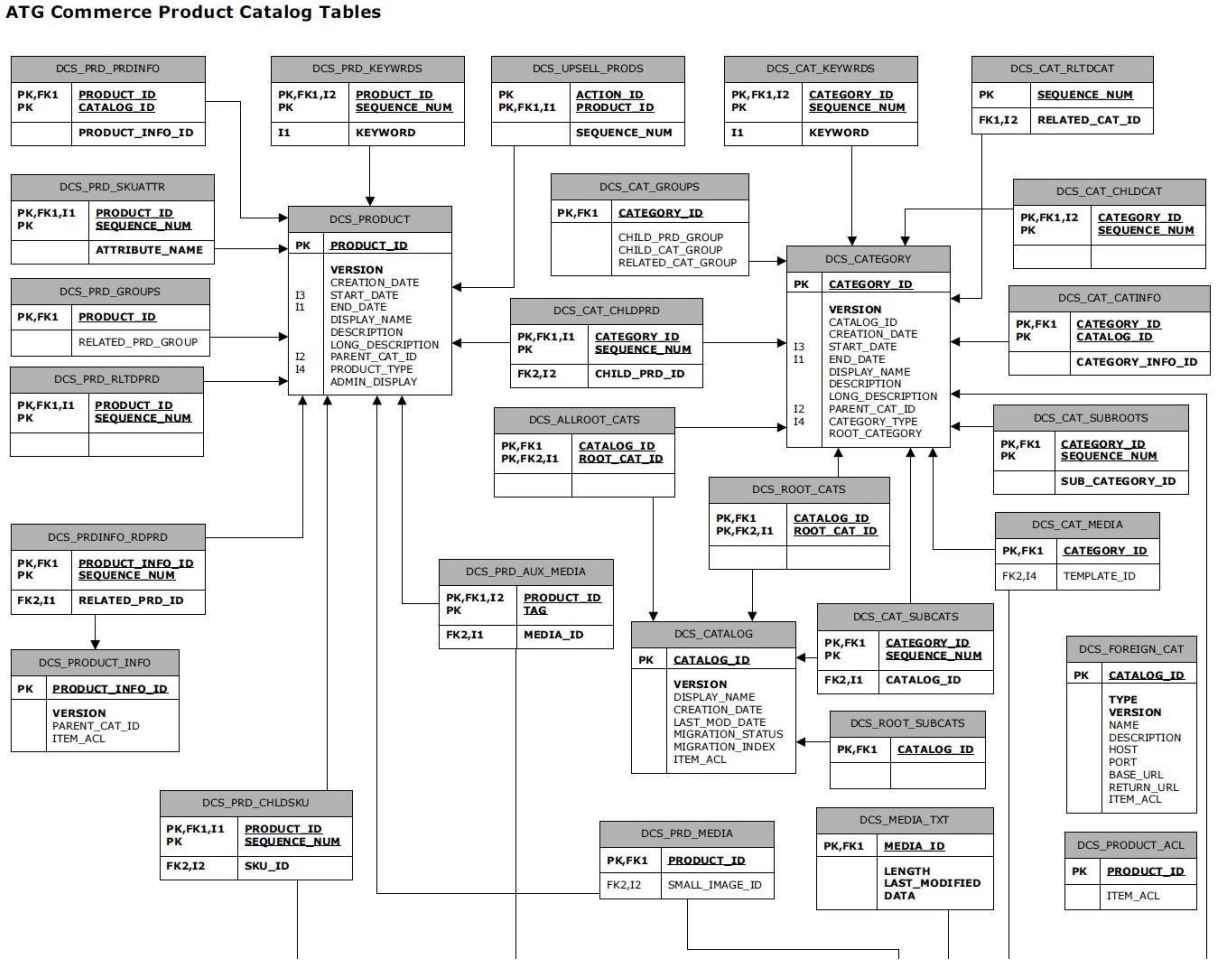
③ 프로젝트–부품

프로젝트	부품
P1	나사
P1	볼트
P2	나사

DB Normalization

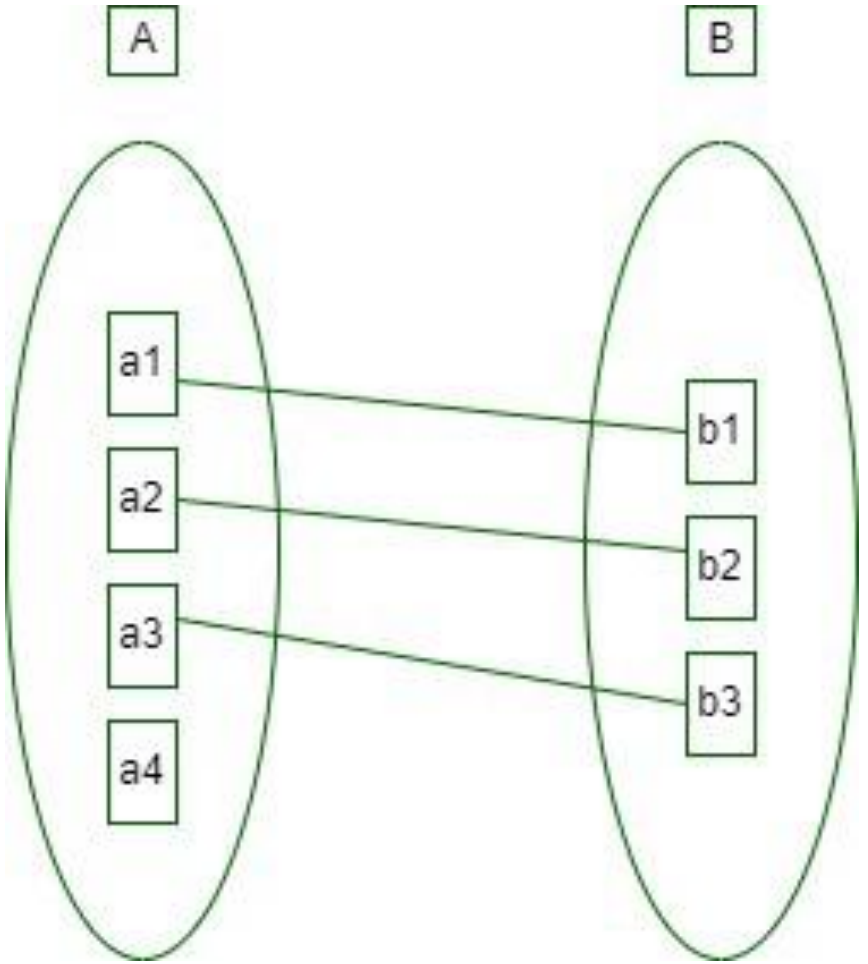
정규화 기법	설명
1차 정규화	원자값 아닌 **도**메인 분해
2차 정규화	**부**분 함수 종속 제거
3차 정규화	**이**행함수 종속 제거
보이스-코드 정규화	**결**정자가 후보 키가 아닌 것 제거
4차 정규화	**다**치종속 제거
5차 정규화	**조**인 종속 제거

정규화의 트레이드오프



과도한 정규화로 테이블이 많아지고 JOIN이 복잡해질 수 있음

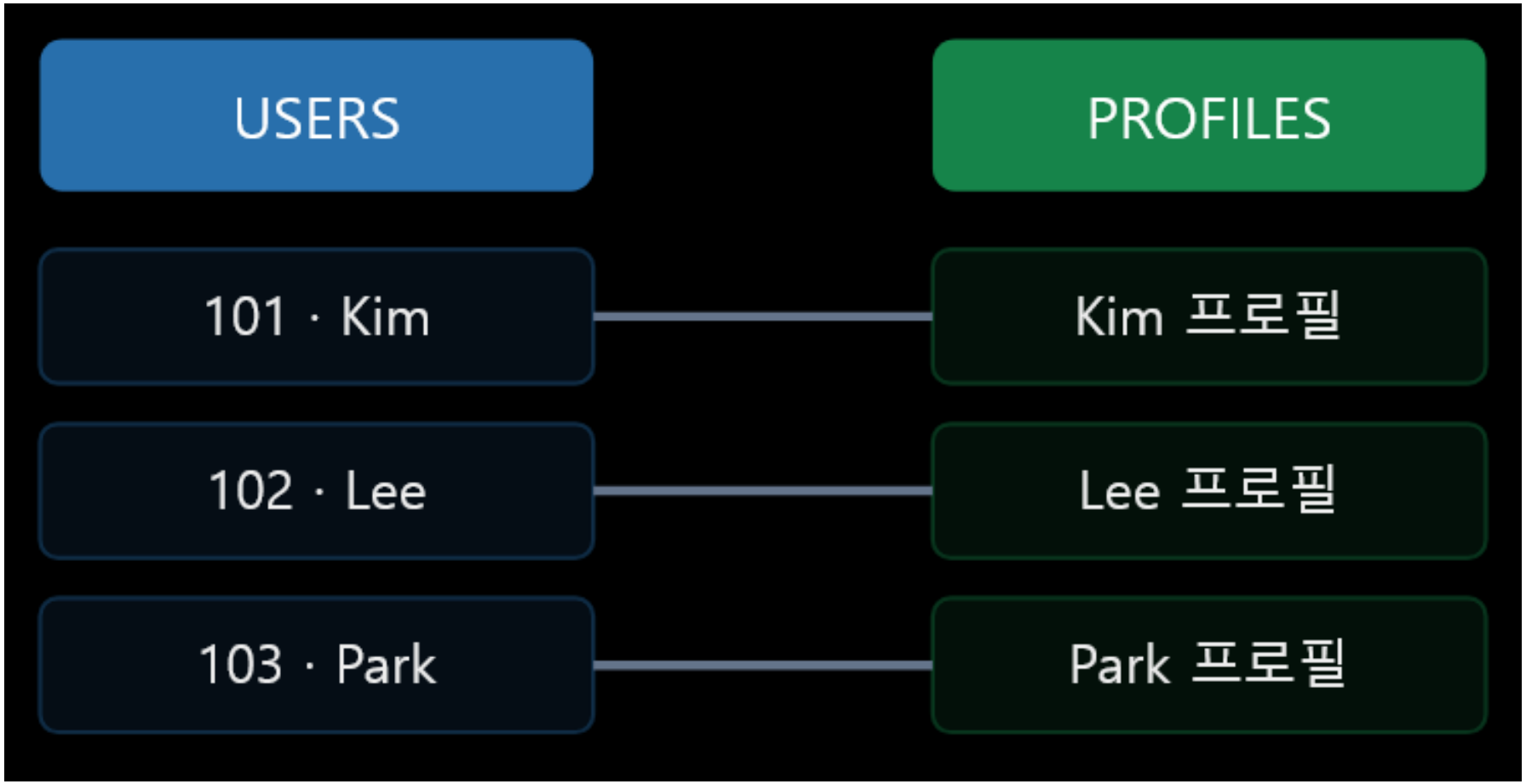
Cardinality



관계형 DB에서 두 테이블의 데이터가

서로 몇 개씩 연결될 수 있는지를 나타내는 관계

1:1



한 사용자는 단 하나의 프로필만 가질 수 있고,

하나의 프로필은 한 사용자에게만 속함

1:N

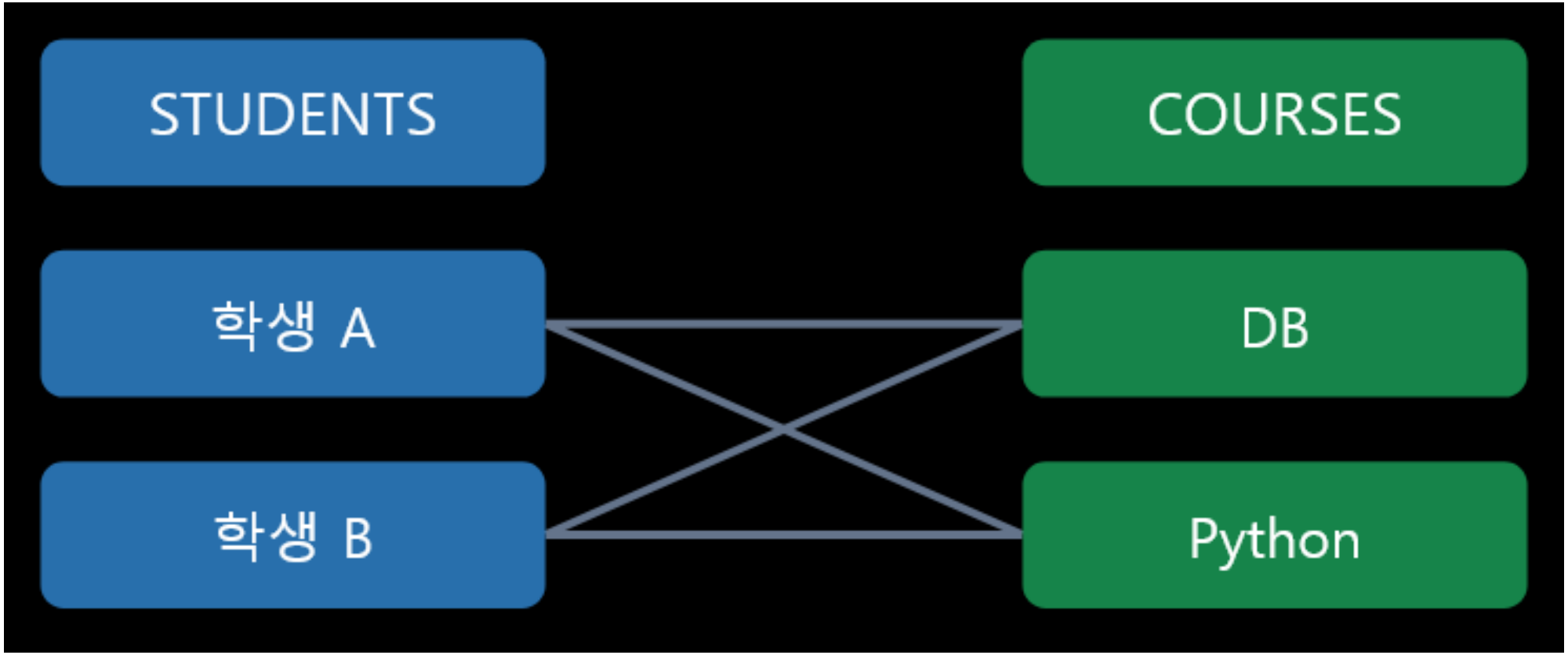


한 고객은 여러 주문이 가능하지만,

각 주문은 하나의 고객에게만 속함

일반적으로 N쪽 테이블에 FK 지정

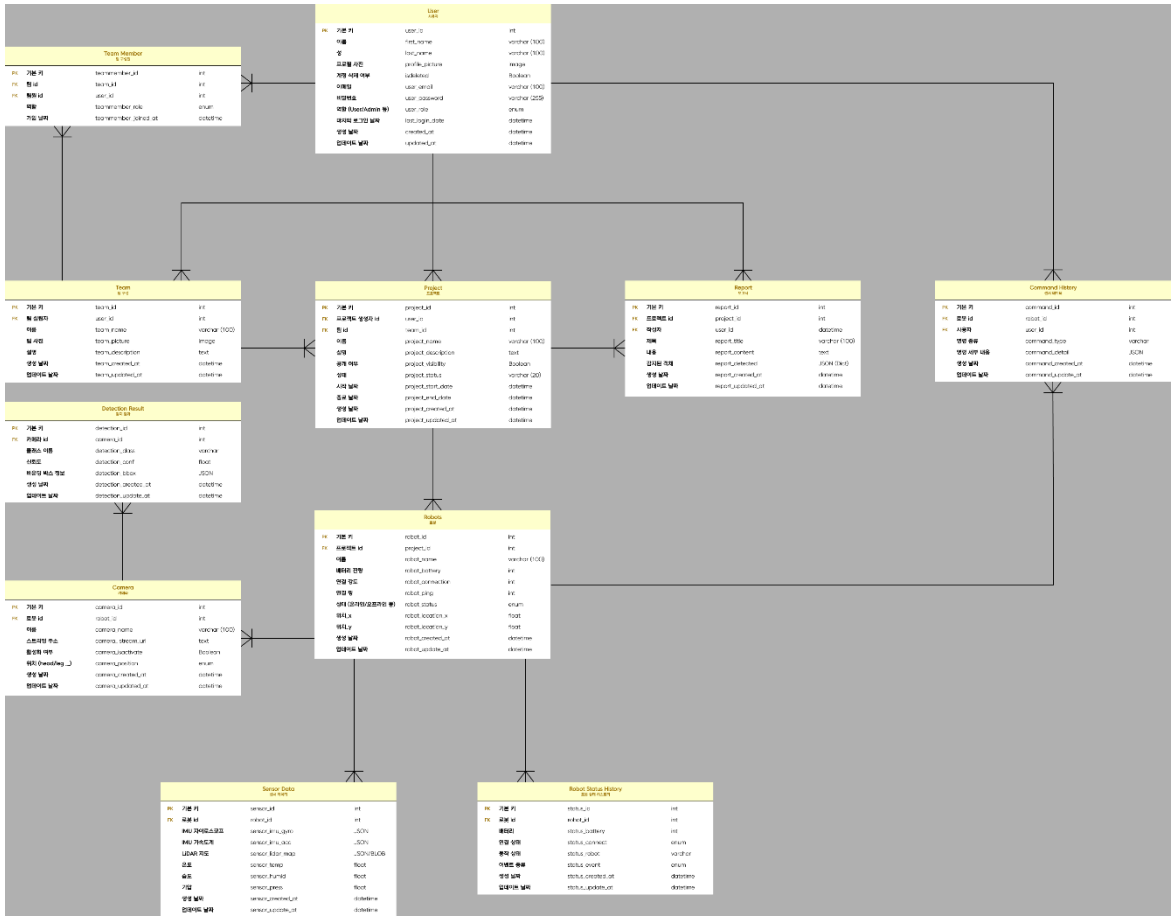
N:M



한 학생은 여러 과목을 수강할 수 있고,

하나의 과목도 여러 학생이 수강할 수 있음

ERD: Entity-Relationship Diagram

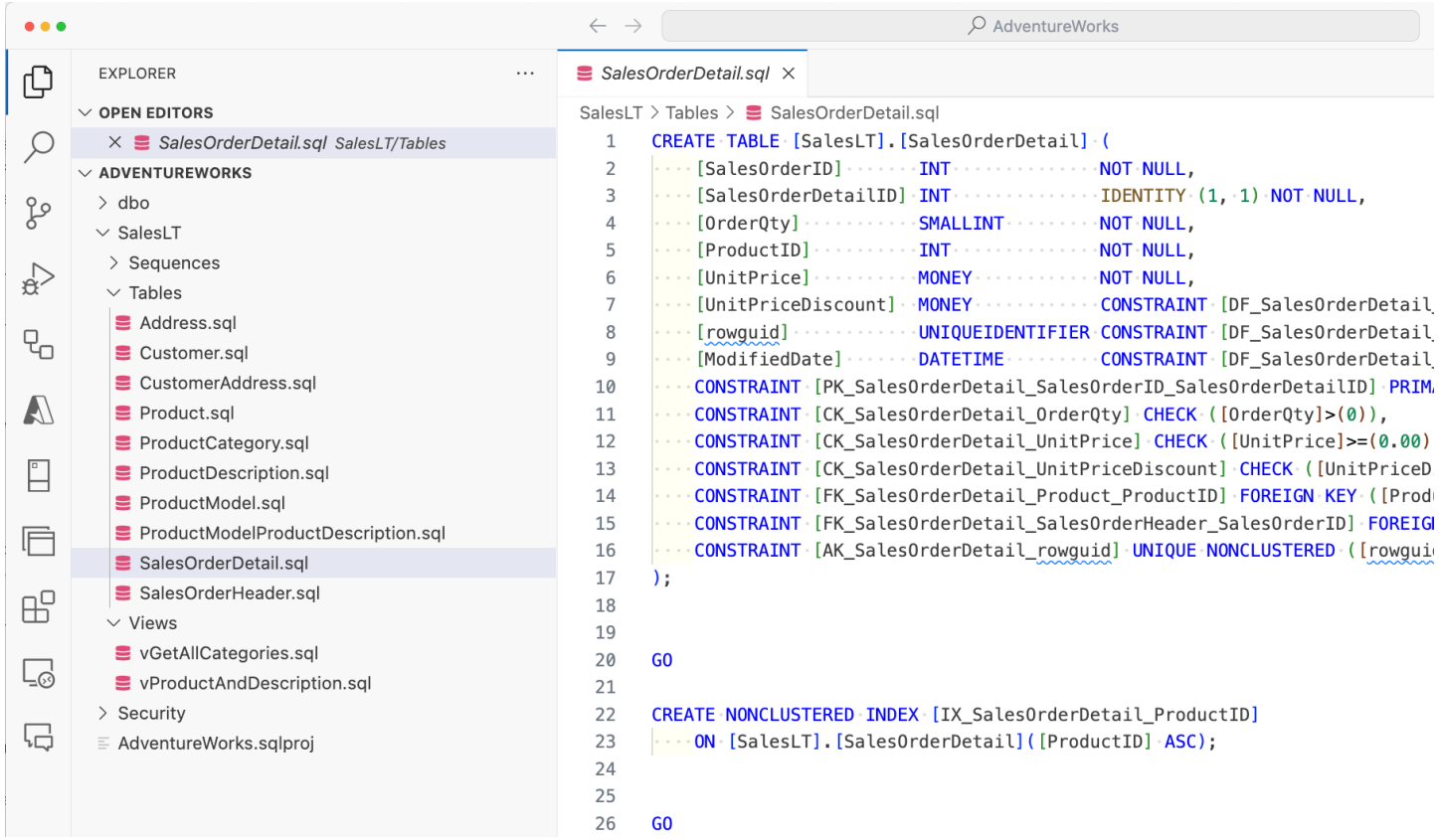


데이터베이스의 테이블, 속성 및 카디날리티 등을 시각적으로 표현한 설계도

02
SQL

SQL

SQL: Structed Query Language



관계형 DB에서 데이터 CRUD 및
DB 구조를 정의하고 관리하는 언어

SQL: Structed Query Language

- SQL문은 대문자로, 테이블 및 컬럼명은 소문자 + _ 로 적는게 일반적임
- SQL문은 한 줄로 쓸 수도, 여러 줄로 쓸 수도 있으나, 하나의 SQL문이 끝나면 세미콜론(;)을 붙여야 함
- 우측의 두 코드는 완벽히 동일

Table: Customers

customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE

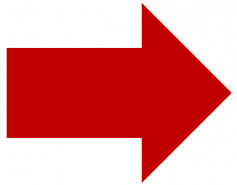
```sql
SELECT * FROM Customer WHERE country = 'USA' ORDER BY age DESC;


SELECT *
FROM Customer
WHERE country = 'USA'
ORDER BY age DESC;
```

SELECT

Table: Customers

customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE



first_name	last_name
John	Doe
Robert	Luna
David	Robinson
John	Reinhardt
Betty	Doe

```
SELECT first_name, last_name FROM Customers;
```

WHERE: 조건문과 비슷함

products			
product_id	**name**	**price**	**stock**
101	Keyboard	30000	10
102	Mouse	15000	20
103	Monitor	200000	5

id	name	price	stock
101	Keyboard	30000	10
103	Monitor	200000	5

```
SELECT * FROM products WHERE price >= 30000;
```

ORDER BY

products

product_id	name	price	stock
101	Keyboard	30000	10
102	Mouse	15000	20
103	Monitor	200000	5

id	name	price	stock
103	Monitor	200000	5
101	Keyboard	30000	10
102	Mouse	15000	20

```
SELECT * FROM products ORDER BY price DESC;
```

INSERT

products			
product_id	**name**	**price**	**stock**
101	Keyboard	30000	10
102	Mouse	15000	20
103	Monitor	200000	5
104	Webcam	50000	8

INSERT INTO products VALUES (104, ‘Webcam’, 50000, 8);

UPDATE

products			
product_id	**name**	**price**	**stock**
101	Keyboard	30000	10
102	Mouse	15000	20
103	Monitor	200000	5
104	Webcam	50000	7

```
UPDATE products SET stock = 7 WHERE product_id = 104;
```

DELETE

products			
product_id	**name**	**price**	**stock**
101	Keyboard	30000	10
102	Mouse	15000	20
103	Monitor	200000	5

```
DELETE FROM products WHERE product_id = 104;
```

GROUP BY

products			
product_id	**name**	**price**	**stock**
101	Keyboard	30000	10
102	Mouse	15000	20
103	Mouse	200000	5
104	Webcam	50000	8

name	total_stock
Keyboard	10
Mouse	25
Webcam	8

```
SELECT name, SUM(stock) AS total_stock
FROM products
GROUP BY name;
```

JOIN: 두 테이블 합치기

VILLAGE

villageid	name
1	Onionville
2	Hilltown

INHABITANT

name	villageid
Alice	1
Bob	1
Charlie	2

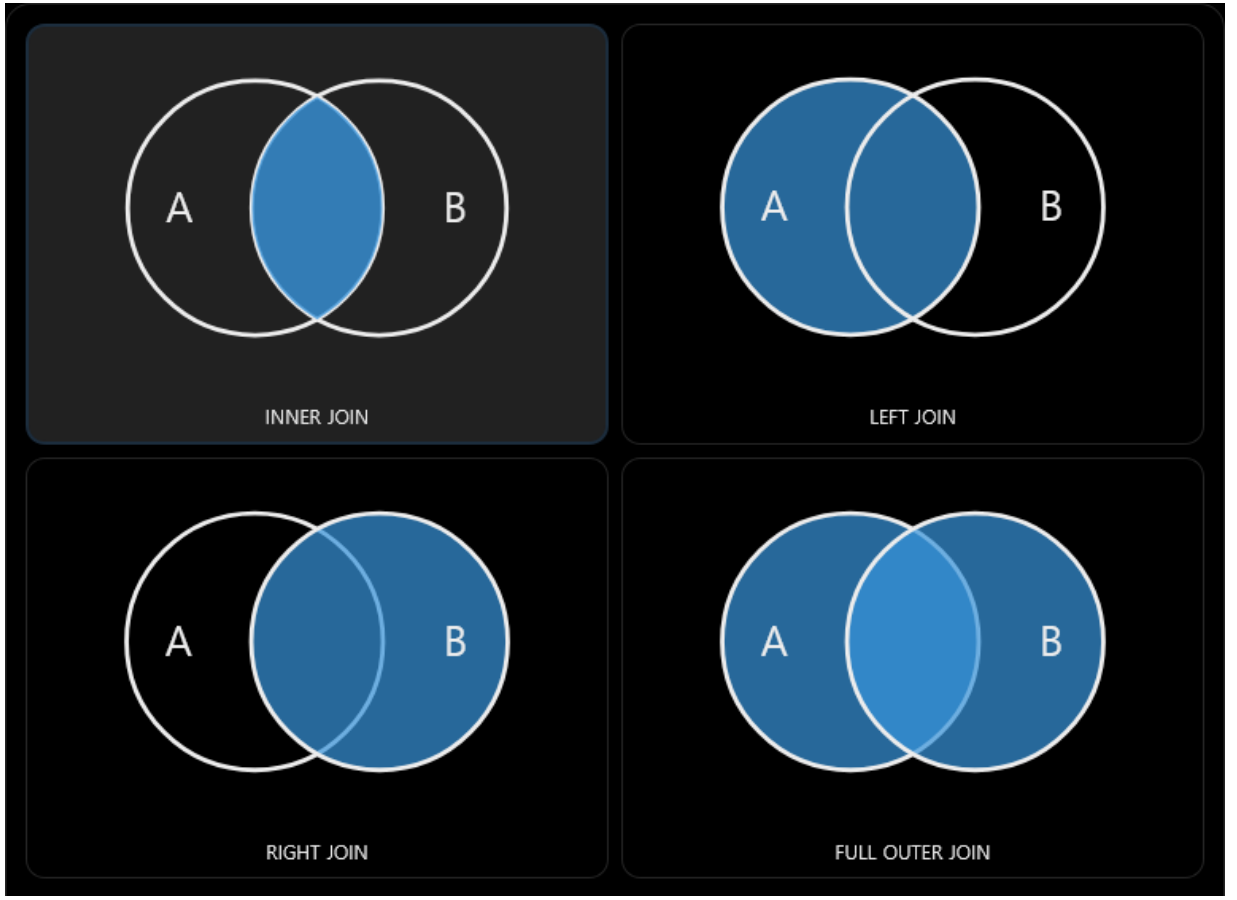
이 두 테이블을 villageid 기준으로 하나로 합치고 싶은 상황

JOIN: 두 테이블 합치기

주민	마을
Alice	Onionville
Bob	Onionville
Charlie	Hilltown

```
SELECT inhabitant.name, village.name
FROM inhabitant
JOIN village ON inhabitant.villageid = village.villageid;
```

JOIN의 종류



예시) SELECT * FROM A INNER JOIN B ON A.id = B.id;

JOIN에서의 검색

VILLAGE		INHABITANT			
villageid	name	name	villageid	gold	job
1	Onionville	Alice	1	100	baker
2	Hilltown	Bob	1	300	merchant
3	Riverwood	Charlie	2	200	baker
		Diana	2	500	merchant
		Eve	3	150	dealer

‘Onionville’에 거주하는 주민의 이름과 돈을 출력하고 싶음
-> JOIN 후 조건문(WHERE)으로 검색

JOIN에서의 검색

name	gold
Alice	100
Bob	300

```sql
SELECT i.name, i.gold
FROM INHABITANT AS i
JOIN VILLAGE AS v
    ON i.villageid = v.villageid
WHERE v.name = 'Onionville';
```

JOIN에서의 검색

VILLAGE

villageid	name
1	Onionville
2	Hilltown
3	Riverwood

INHABITANT

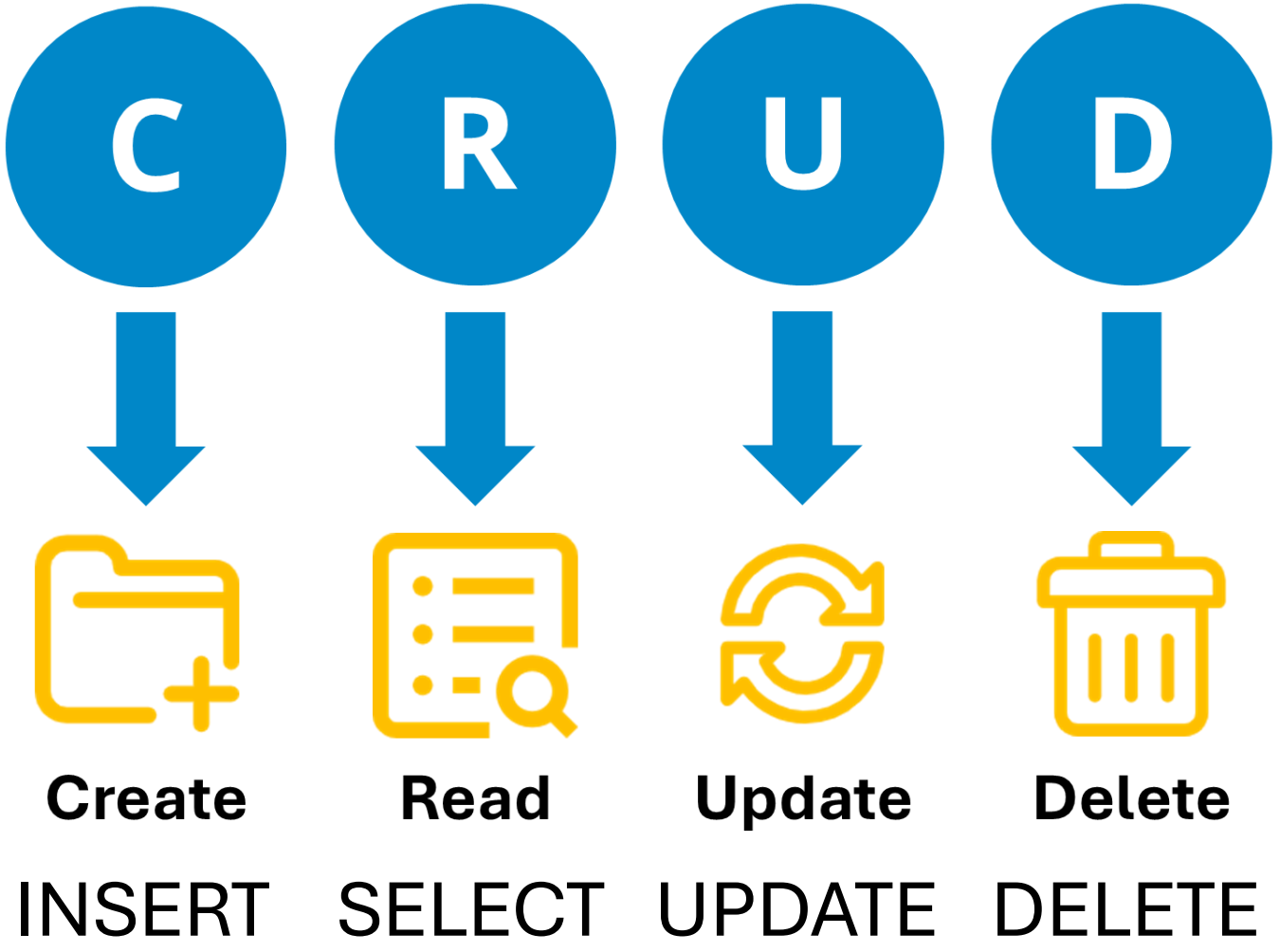
name	villageid	gold	job
Alice	1	100	baker
Bob	1	300	merchant
Charlie	2	200	baker
Diana	2	500	merchant
Eve	3	150	dealer

이번에는 마을별 주민들의 돈 총합과
그 중 합계가 300 이상인 마을만 출력하고 싶음

기타 기초 문법

COUNT()	SUM()	AVG()	MAX() / MIN()
DISTINCT	HAVING	LIMIT	AND / OR / NOT
IN	BETWEEN	LIKE	IS NULL / IS TRUE
CASE WHEN	COALESCE()	UNION	IS NOT NULL

CRUD


C
R
U
D
Create
Read
Update
Delete
INSERT
SELECT
UPDATE
DELETE

03
인덱스 & 검색

Index & Search

Sequential Search

```sql
SELECT *
FROM products
WHERE price = 30000;
```

만약 상품이 100만개라면 가격이 30,000원인
상품을 찾기 쉽지 않을듯함

Sequential Search

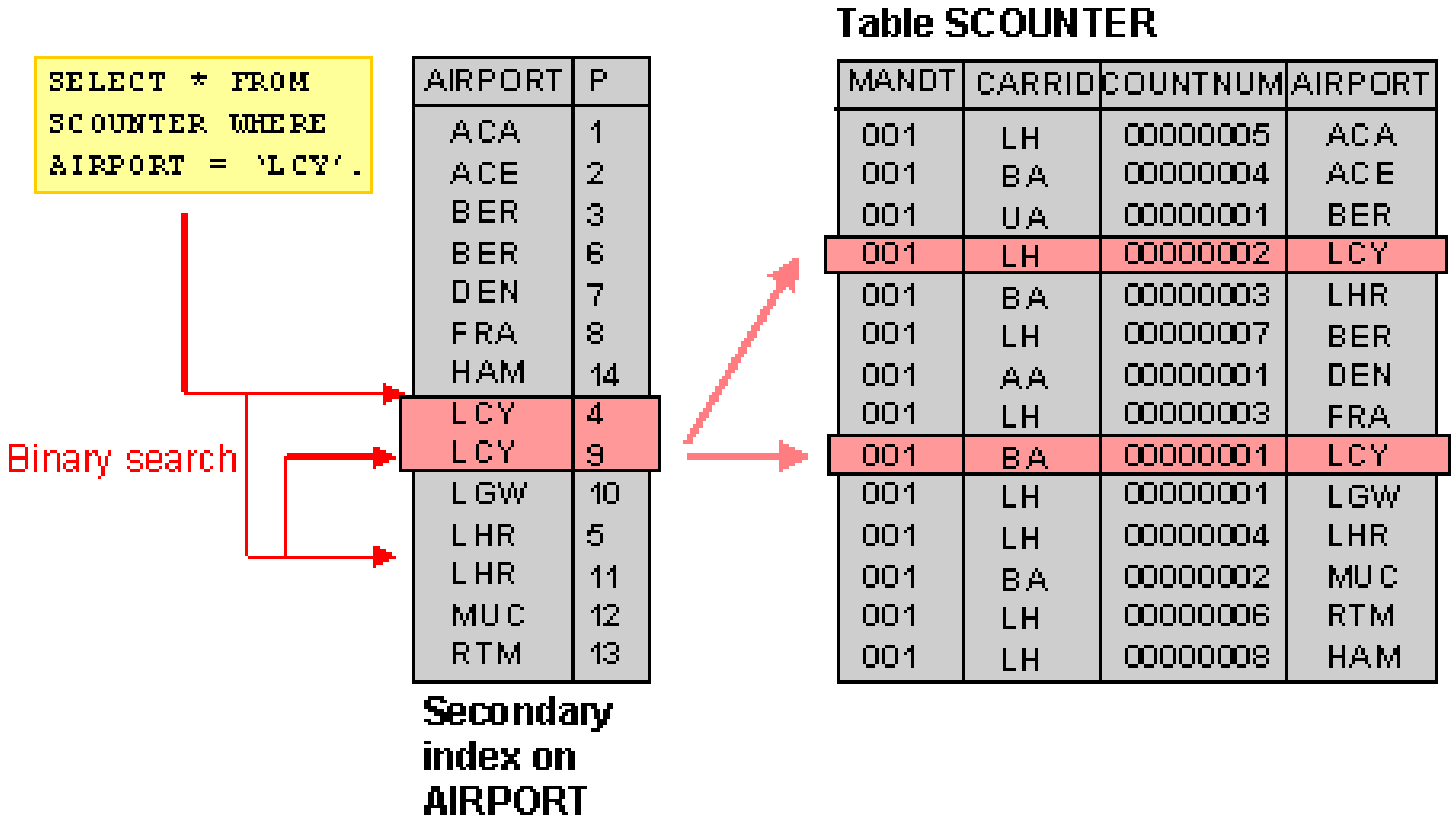
products 테이블 Index 없음

product_id	name	price	상태
101	Keyboard	30,000	✓
102	Mouse	15,000	검사 중
103	Monitor	200,000	—
104	Webcam	50,000	—

Sequential Search

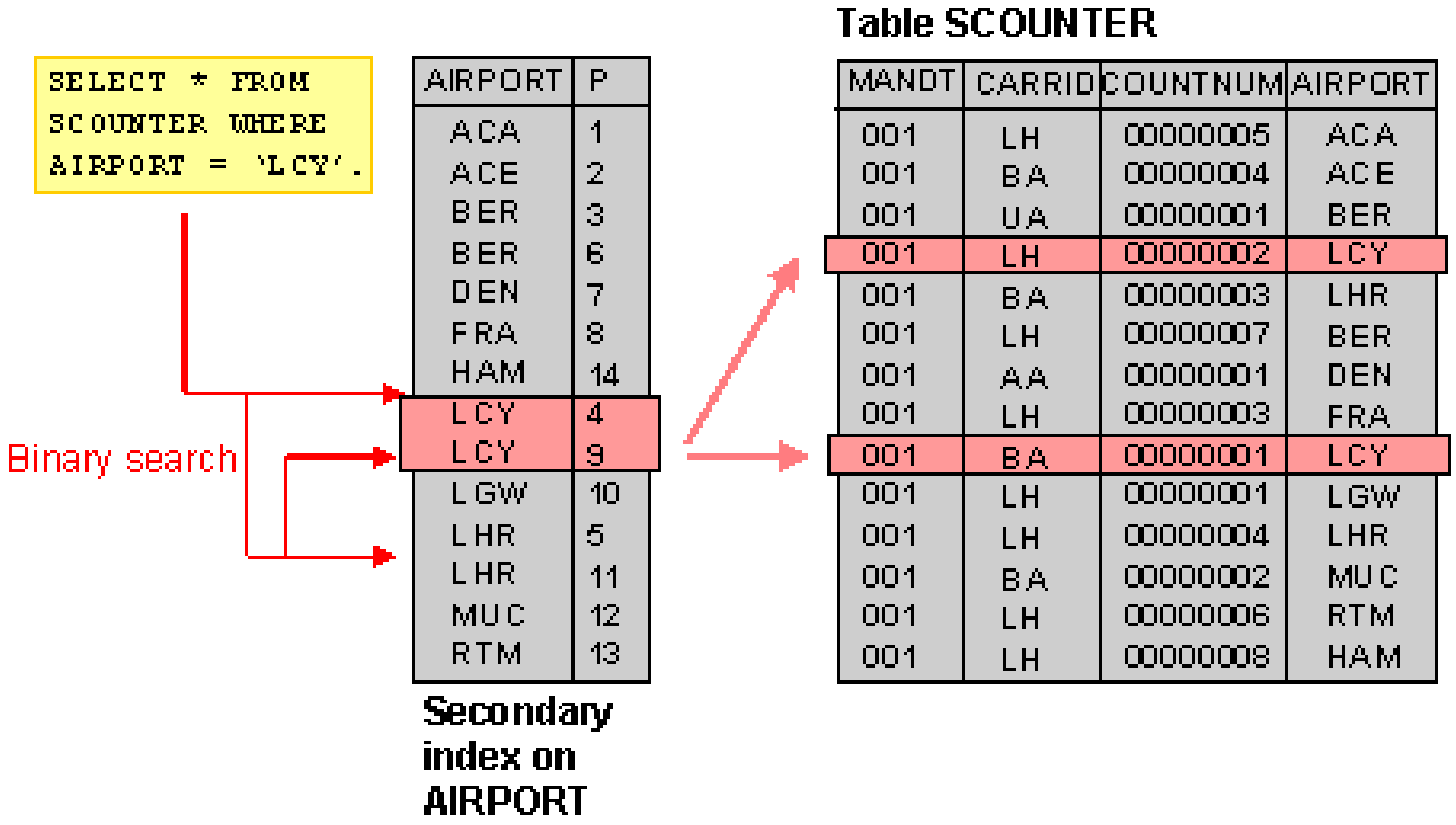
가장 단순하게 위에서부터 하나하나 검사

DB Index



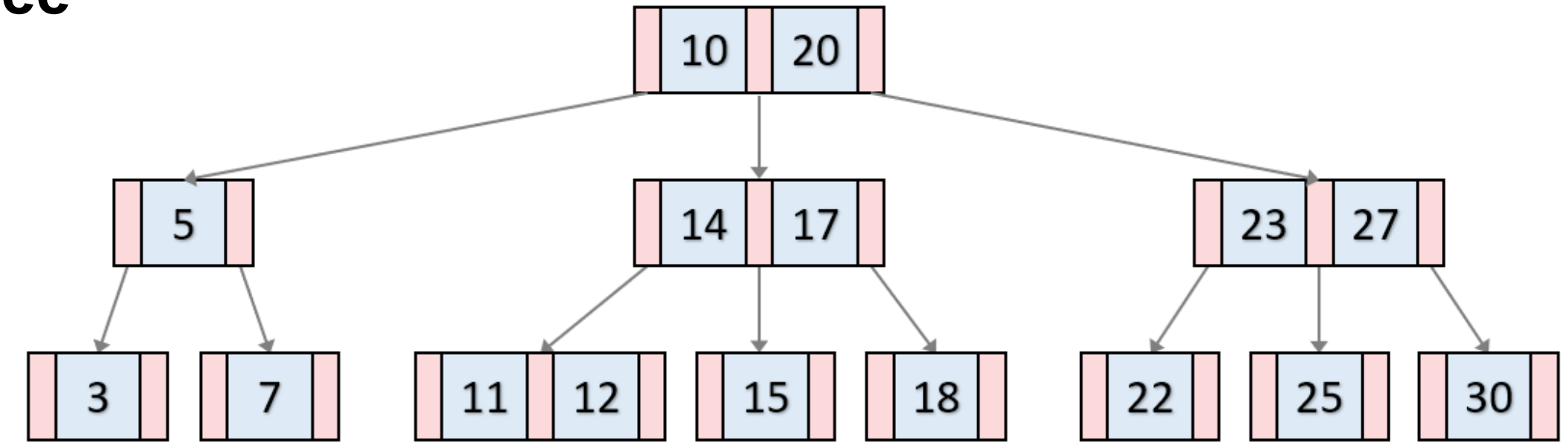
Secondary Index 테이블에는 AIRPORT를 정렬해 놓았기 때문에 LCY를 빠르게 찾을 수 있음 (이진탐색으로 탐색)

DB Index



그러나 검색 속도와 데이터의 삽입 및 삭제 시의 유지비용 때문에
데이터가 많아지고 자주 변경될수록 테이블형 인덱스는 잘 쓰이지 않음

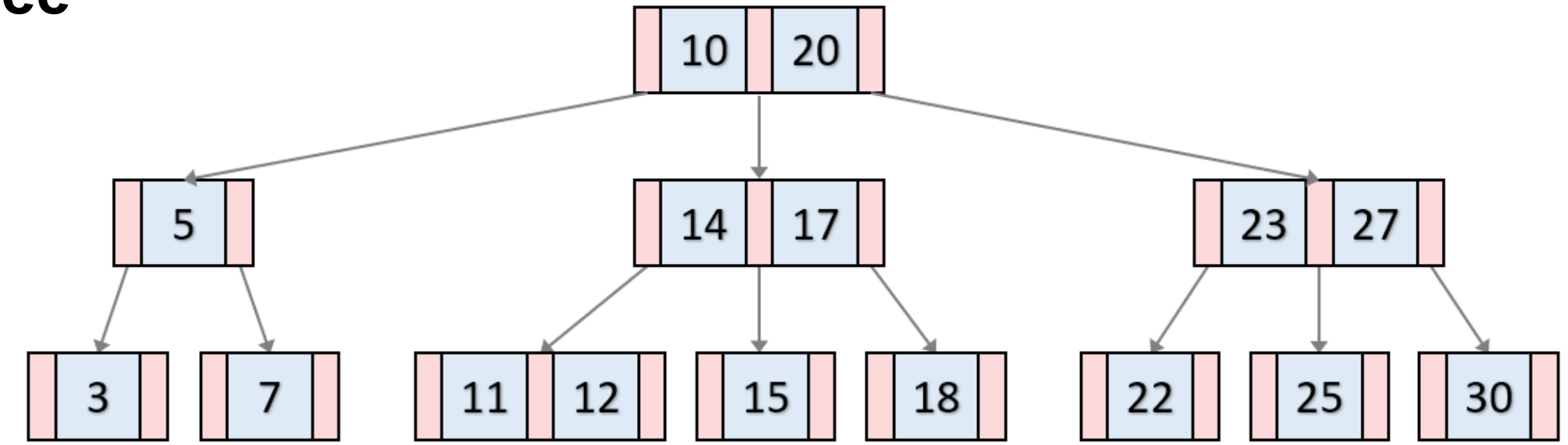
B-Tree



- 위는 3차 B트리를 나타낸 그림

- 파란 부분은 각 노드의 Key, 빨간 부분은 자식 노드들을 가리키는 포인터

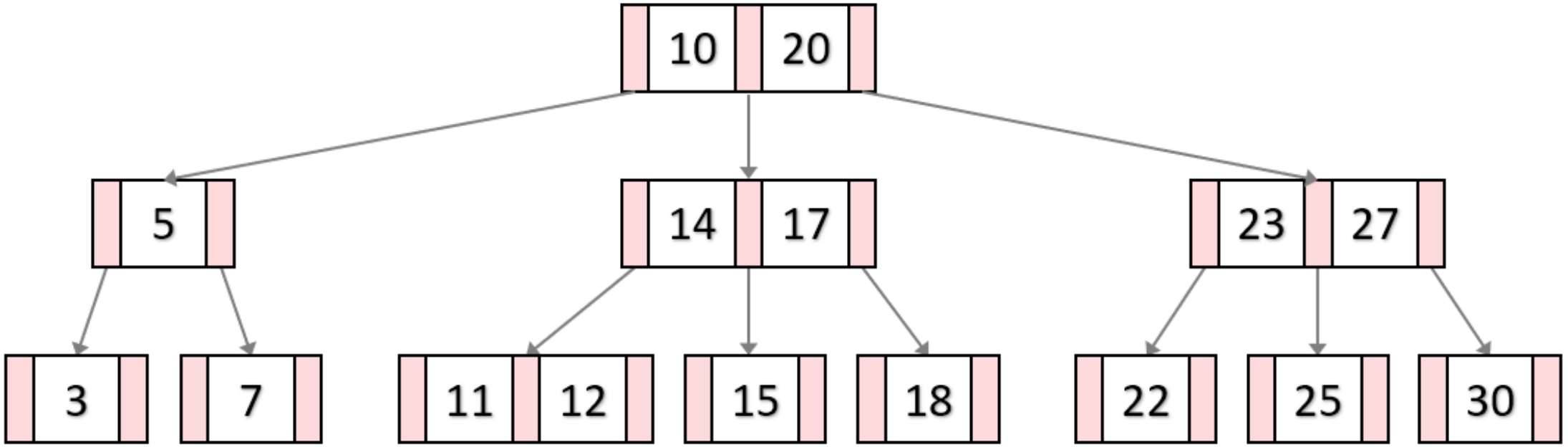
B-Tree



- 최대 M개의 자식을 가질 수 있는 B트리를 M차 B트리라고 함
  - 노드는 최소 M/2개, 최대 M개의 자식을 가질 수 있음
- 노드의 키(노드 안의 데이터)가 x개라면 자식은 x + 1개를 가짐
  - 노드 안의 키들은 큰 값이 오른쪽으로 오도록 정렬됨

B-Tree의 검색 과정

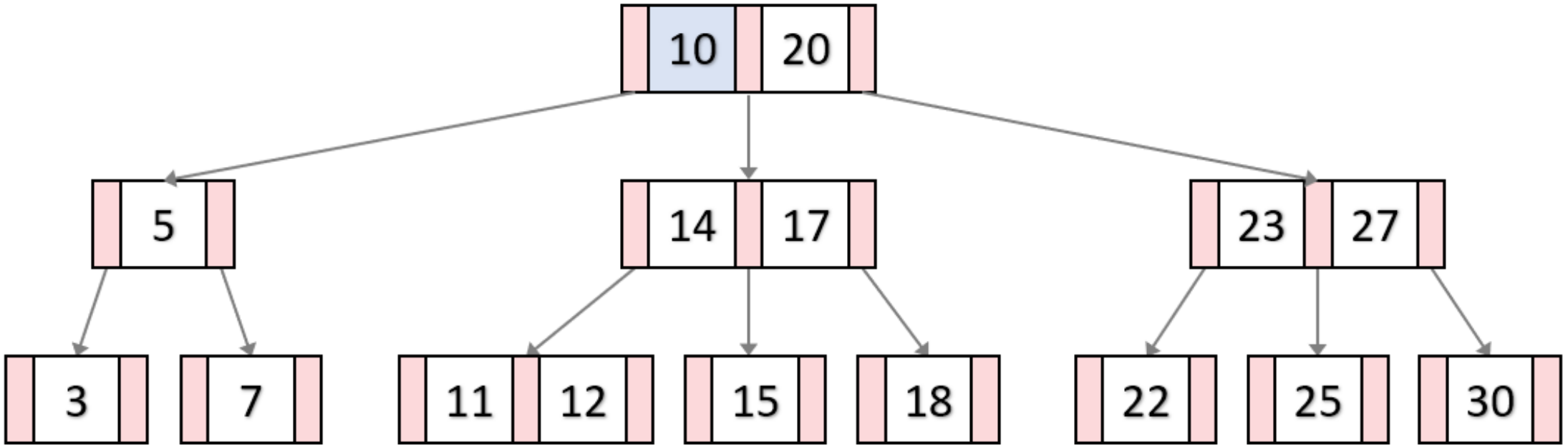
1 18 검색 시작, 루트노드의 key를 순회하면서 검색 시작



B트리에서 18이라는 값 검색

B-Tree의 검색 과정

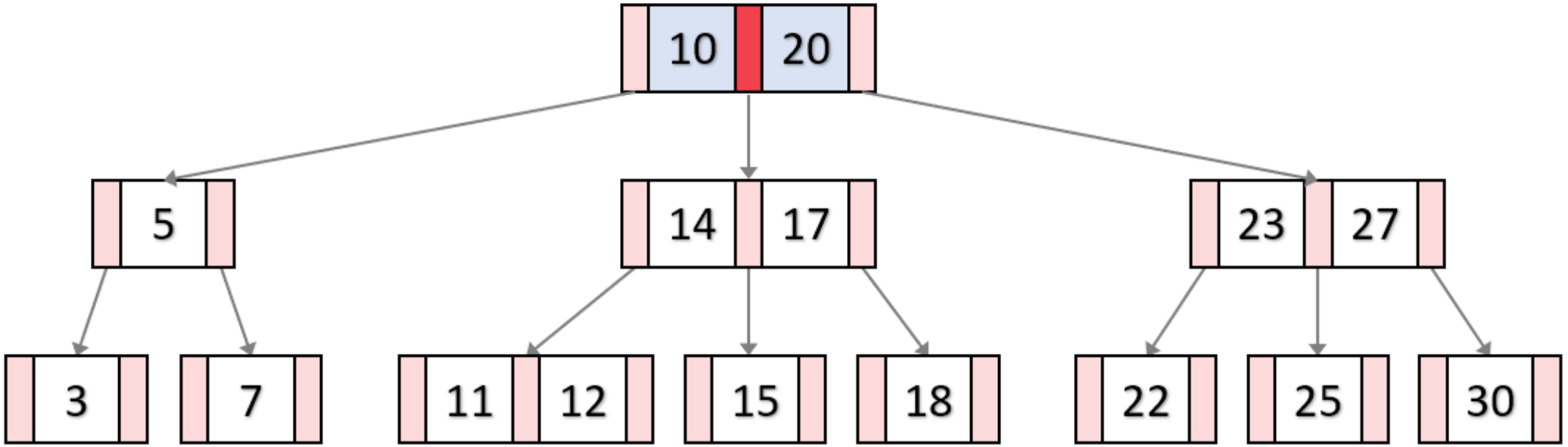
2 18은 10보다 크기 때문에 다음 key를 검사



B트리에서 18이라는 값 검색

B-Tree의 검색 과정

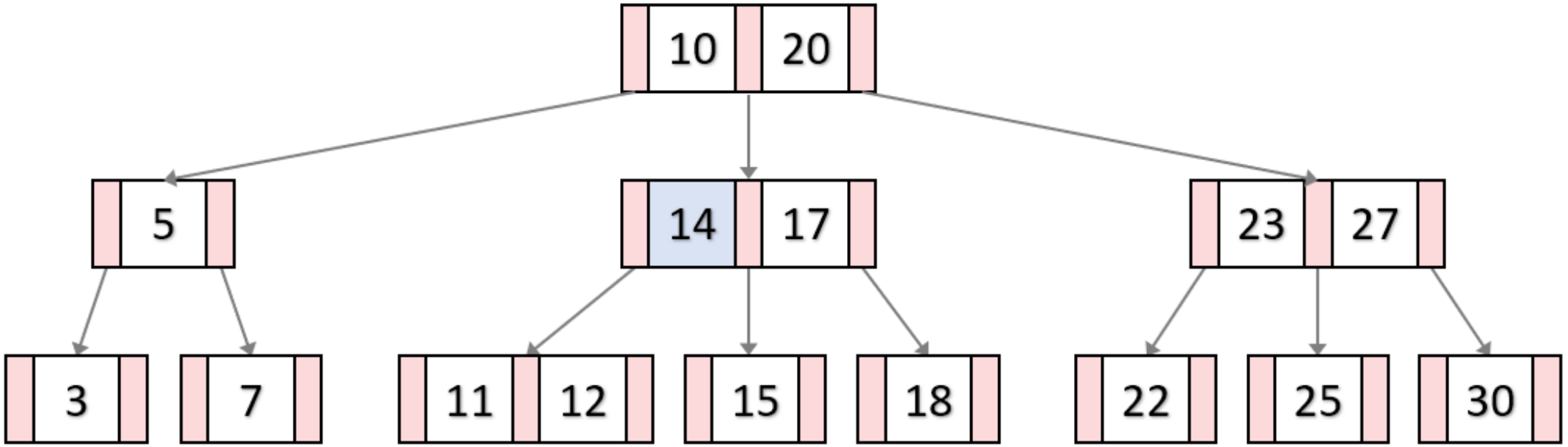
3 18은 10보다 크고, 20보다 작기 때문에 10과 20 사이의 자식 노드로 이동



B트리에서 18이라는 값 검색

B-Tree의 검색 과정

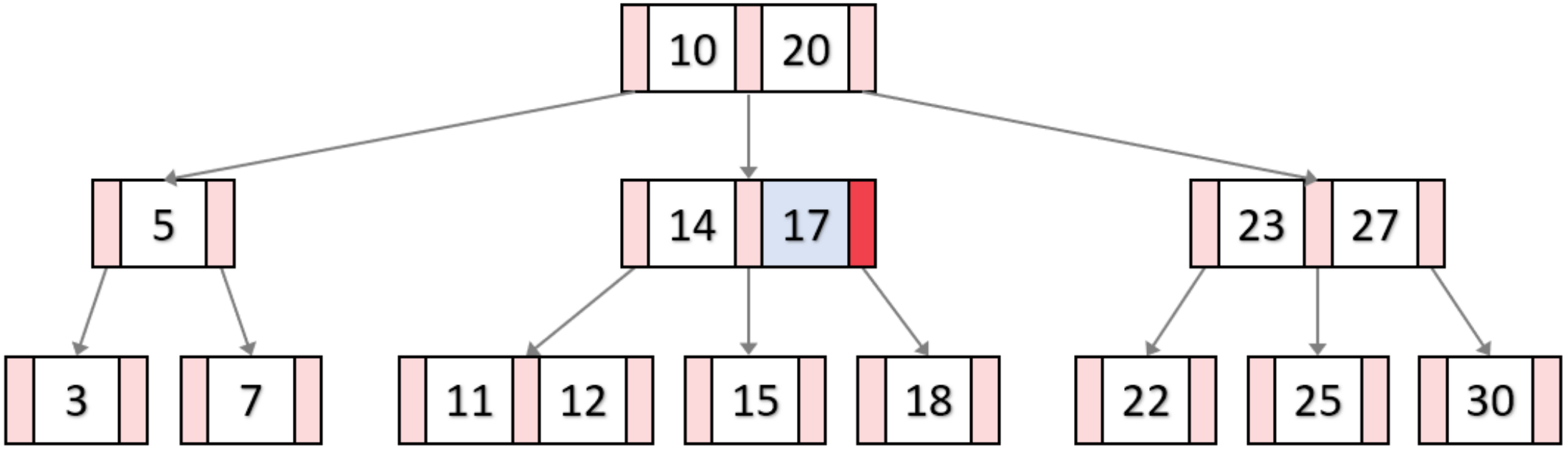
4 자식노드에서 다시 검색 시작. 18은 14보다 크기 때문에 다음 key를 검사



B트리에서 18이라는 값 검색

B-Tree의 검색 과정

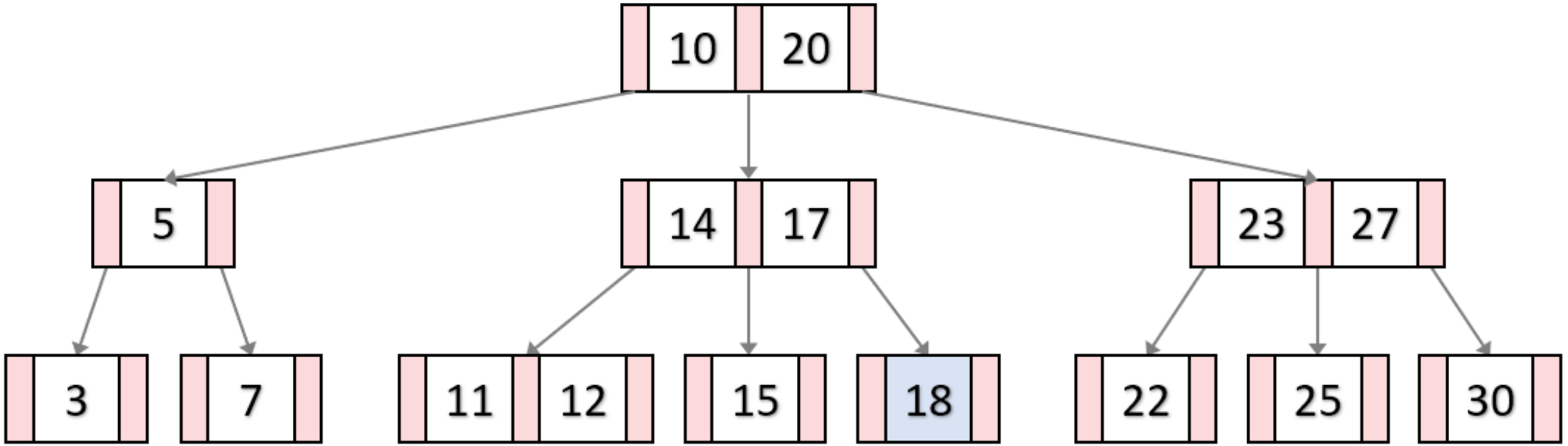
5 18은 노드의 가장 마지막 key인 17보다 크기 때문에 노드의 가장 마지막 자식노드로 이동



B트리에서 18이라는 값 검색

B-Tree의 검색 과정

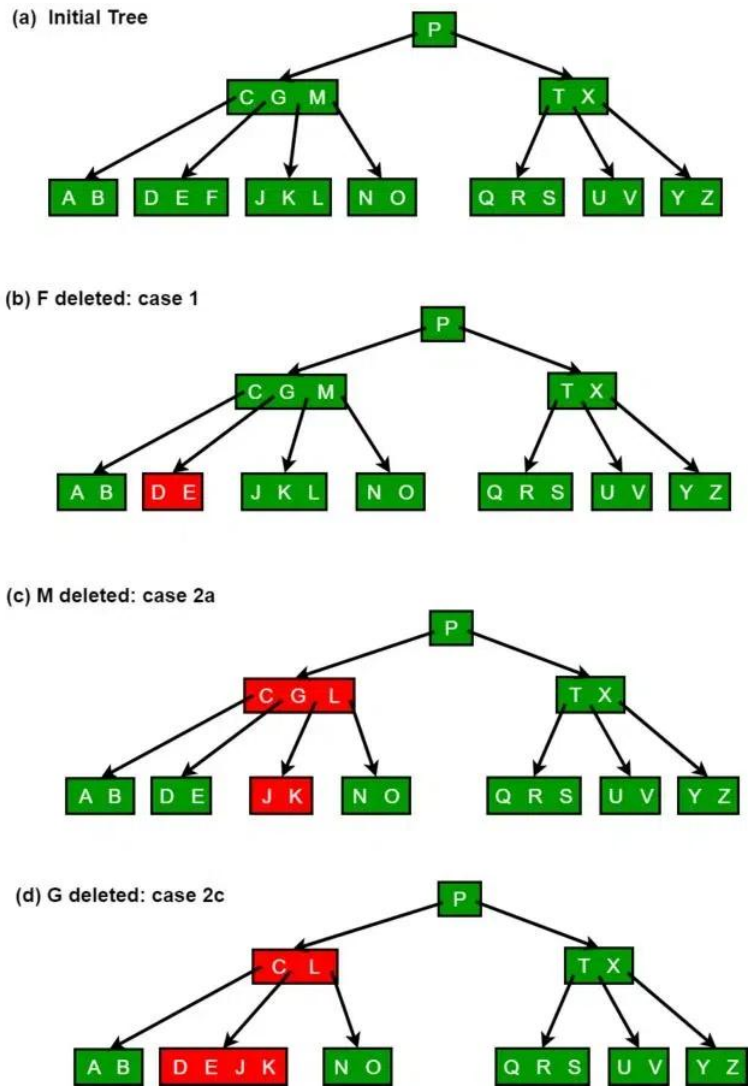
6 18 검색 완료



B트리에서 18이라는 값 검색

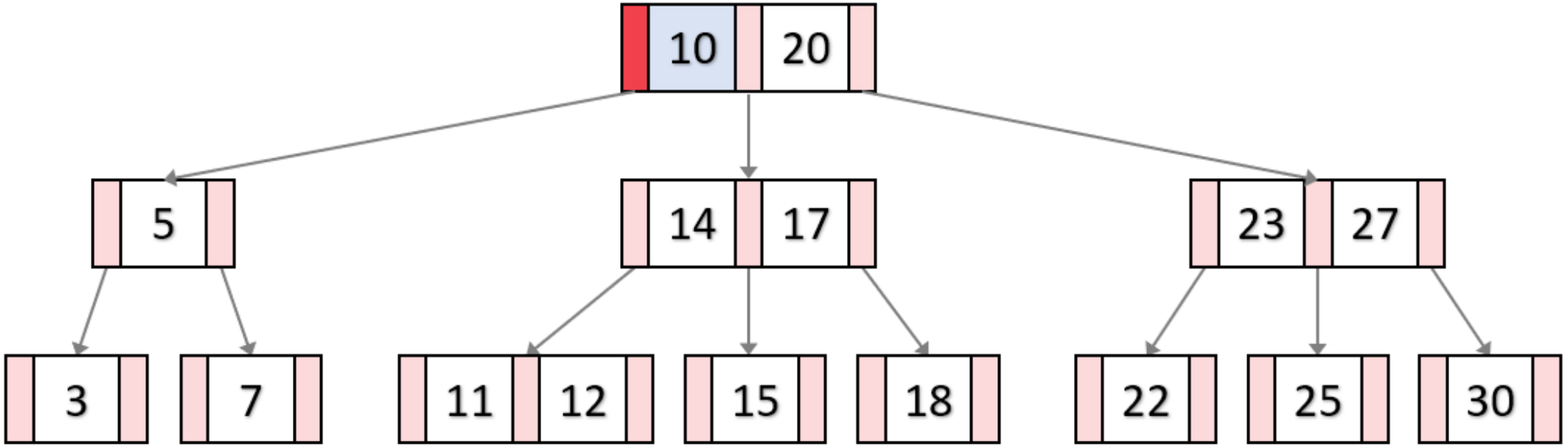
B-Tree의 삽입 과정

- 요소가 삽입되기에 적절한 리프 노드를 검색하고, 필요한 경우 노드 분할
- 삽입 시 노드가 분할되는 경우와 분할이 일어나지 않는 경우로 나뉨



B-Tree의 삽입 과정 (분할 X)

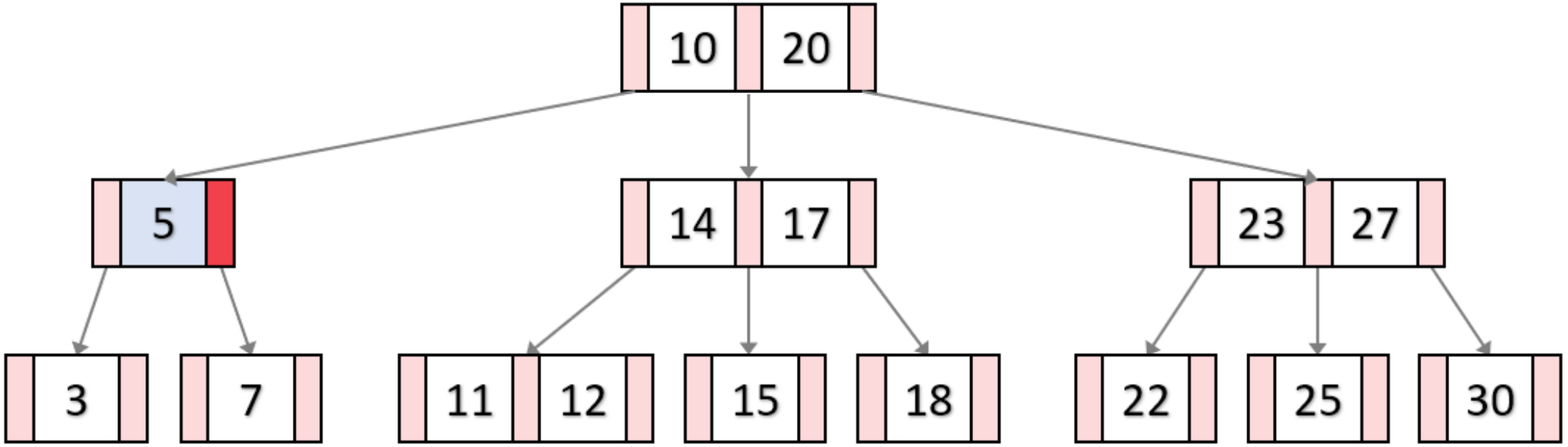
1 9 삽입 시작, 루트노드에서 key 10이 9보다 크기 때문에 가장 왼쪽 자식노드로 이동



B트리에 9 삽입

B-Tree의 삽입 과정 (분할 X)

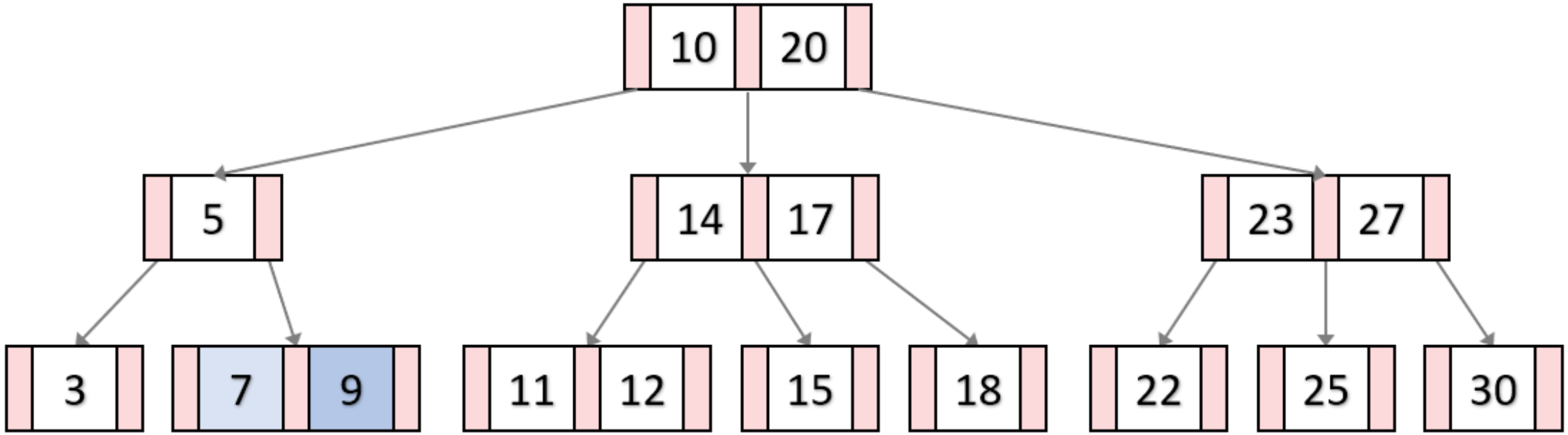
2 9는 5보다 크기 때문에 가장 오른쪽 자식노드로 이동



B트리에 9 삽입

B-Tree의 삽입 과정 (분할 X)

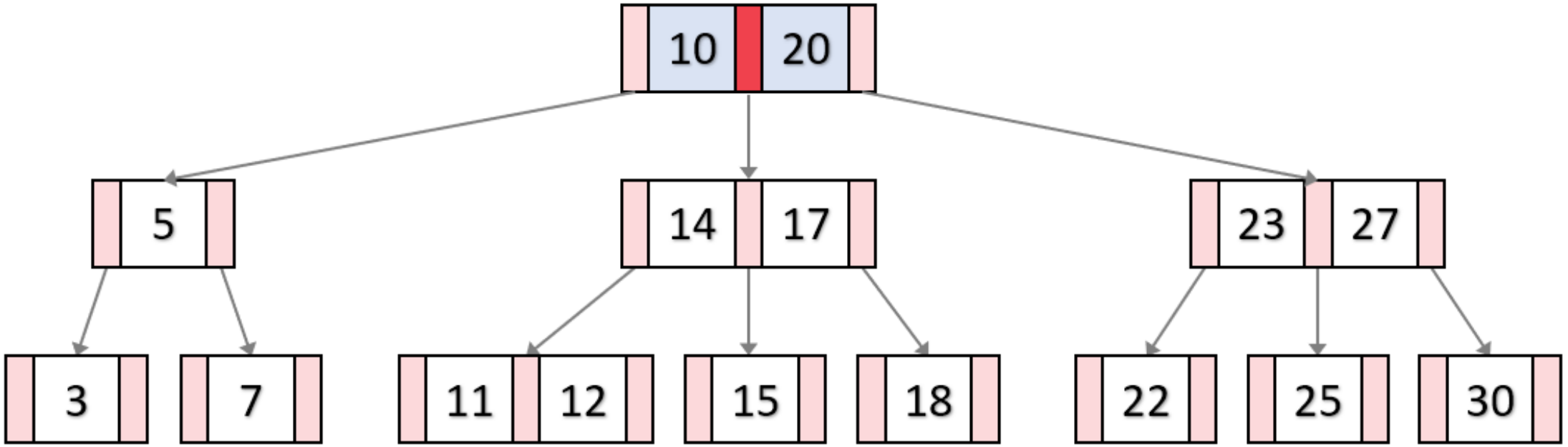
3 리프노드에 도달, 9는 7보다 크기 때문에 7 오른쪽에 9 삽입 완료



B트리에 9 삽입

B-Tree의 삽입 과정 (분할 O)

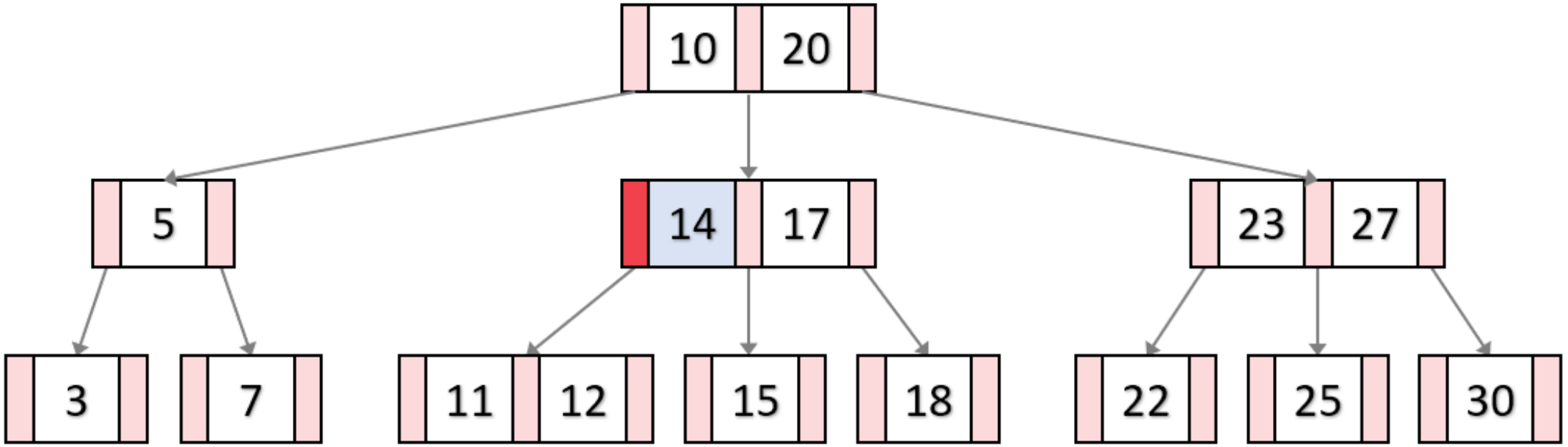
1 13 삽입 시작, 루트노드의 key 중 10보다 크고 20보다 작기 때문에 중간 자식노드로 이동



B트리에 13 삽입

B-Tree의 삽입 과정 (분할 O)

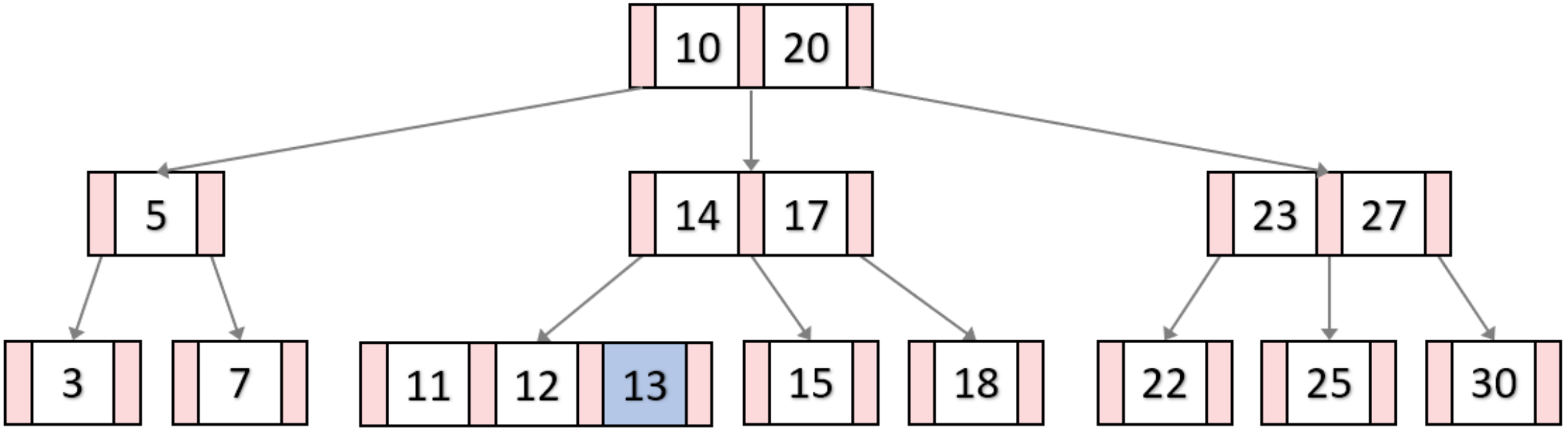
2 13은 14보다 작기 때문에, 가장 왼쪽 자식노드로 이동



B트리에 13 삽입

B-Tree의 삽입 과정 (분할 O)

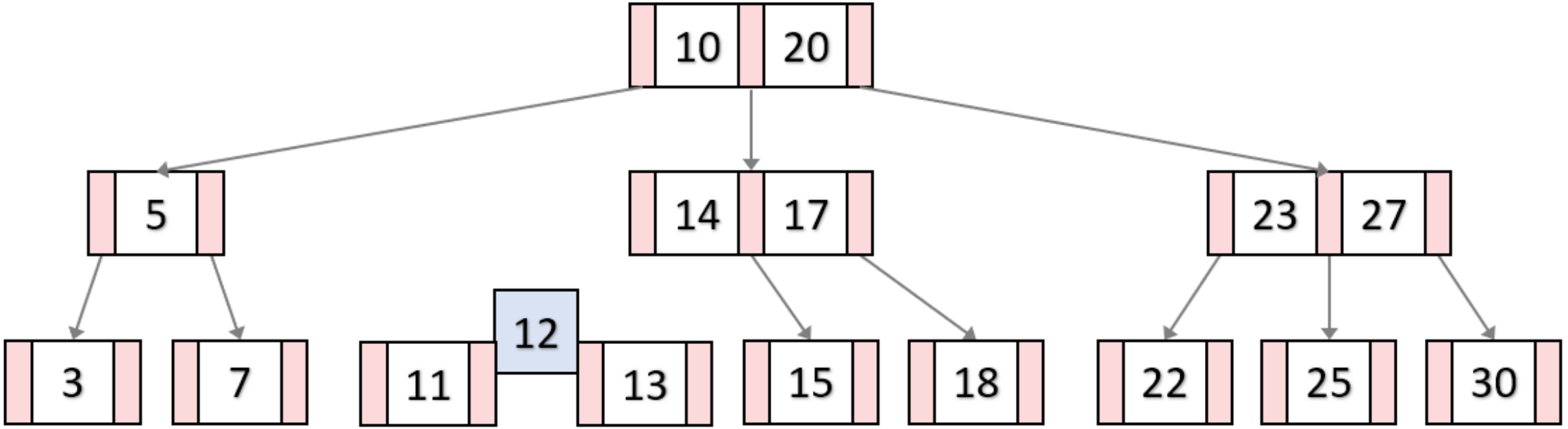
3 13은 12보다 크기 때문에 가장 마지막에 삽입. 해당 노드가 최대로 가질 수 있는 key의 개수를 초과했기 때문에 분할을 수행



B트리에 13 삽입

B-Tree의 삽입 과정 (분할 O)

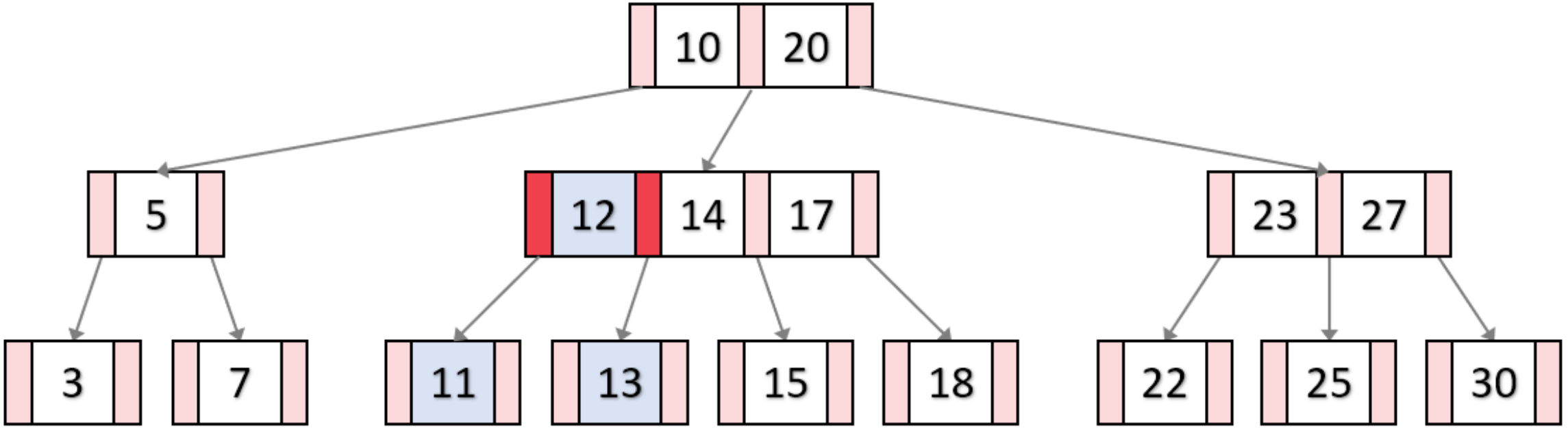
4 중앙값 12를 기준으로 분할을 수행



B트리에 13 삽입

B-Tree의 삽입 과정 (분할 O)

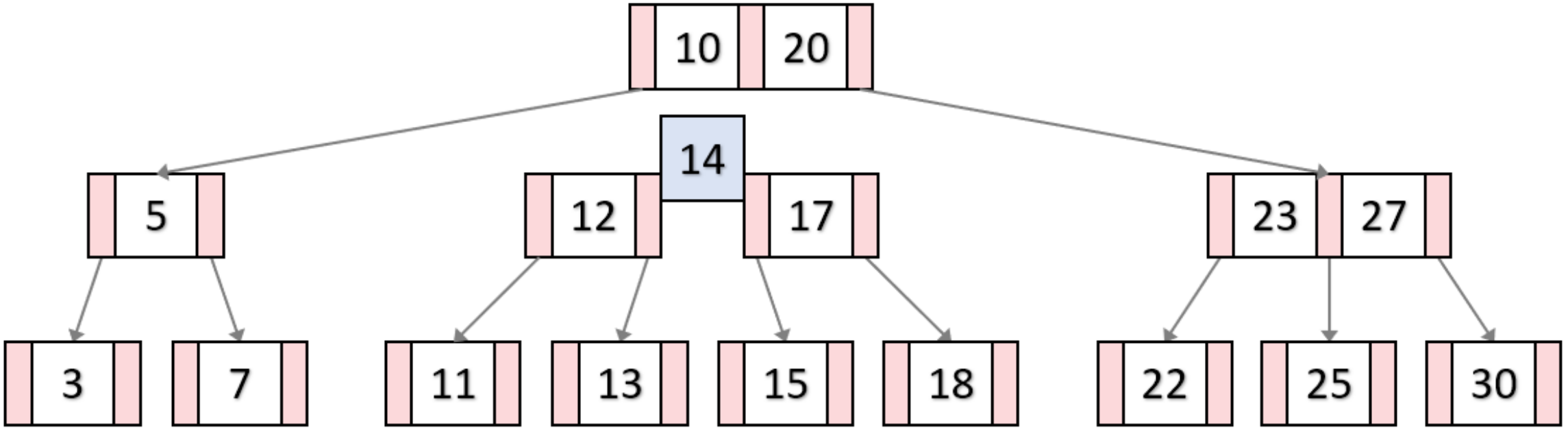
5 12는 부모노드에 오름차순으로 삽입, 11과 13은 12의 왼쪽자식, 오른쪽 자식으로 설정



B트리에 13 삽입

B-Tree의 삽입 과정 (분할 O)

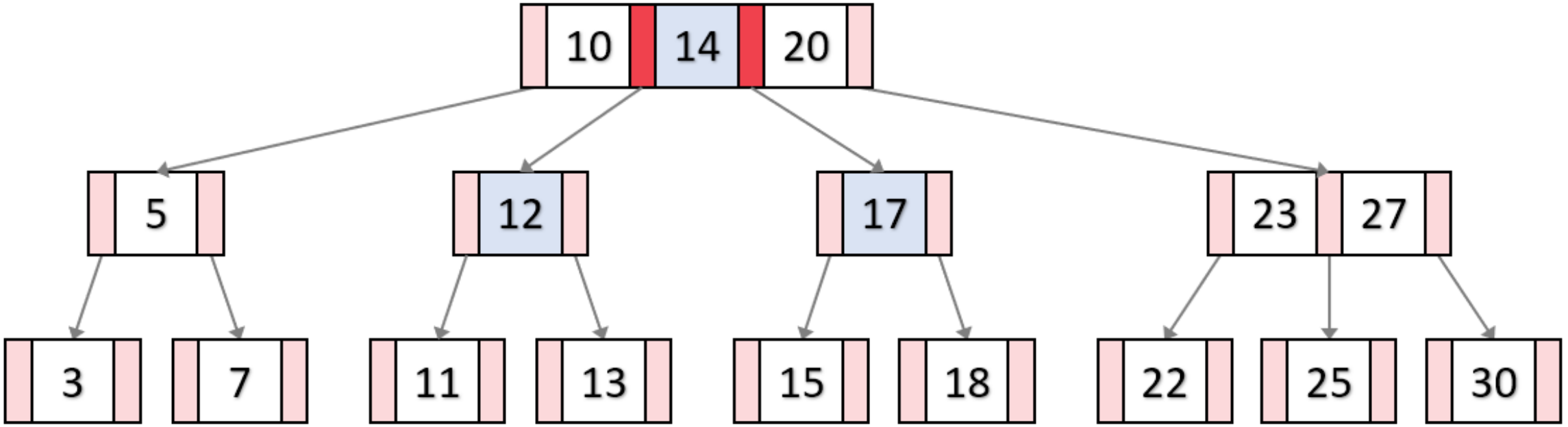
6 12가 병합된 노드가 최대로 가질 수 있는 key의 개수를 초과, 중앙값 14를 기준으로 분할을 수행



B트리에 13 삽입

B-Tree의 삽입 과정 (분할 O)

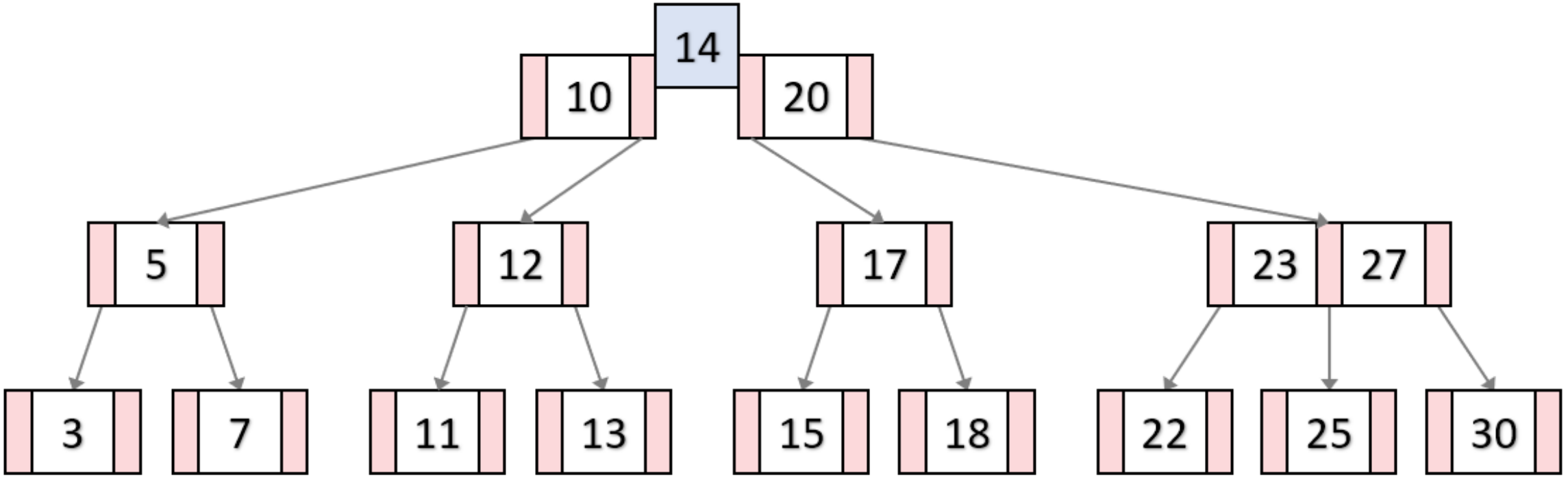
7 14는 부모노드에 오름차순으로 삽입, 12과 17은 14의 왼쪽자식, 오른쪽 자식으로 설정



B트리에 13 삽입

B-Tree의 삽입 과정 (분할 O)

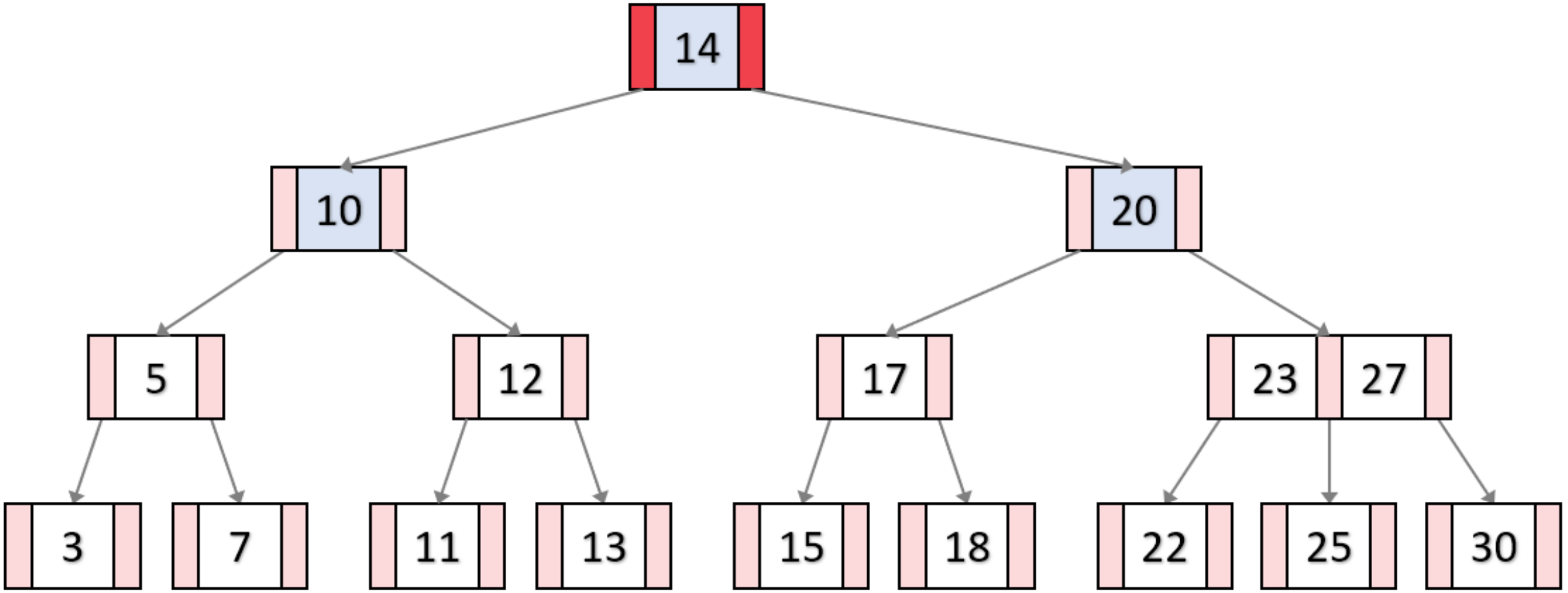
8 14가 병합된 노드가 최대로 가질 수 있는 key의 개수를 초과, 중앙값 14를 기준으로 분할을 수행



B트리에 13 삽입

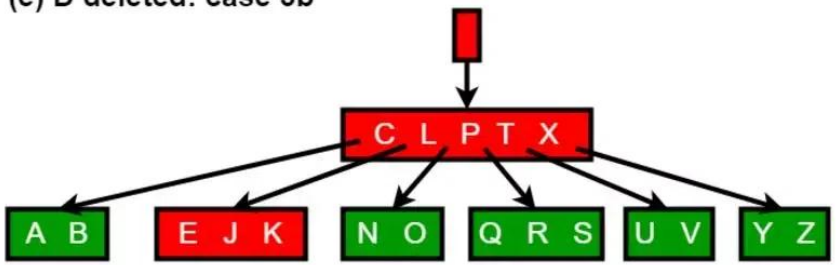
B-Tree의 삽입 과정 (분할 O)

9 14가 새로운 루트노드가 되고, 10과 20은 14의 왼쪽자식, 오른쪽 자식으로 설정. 삽입과정 완료

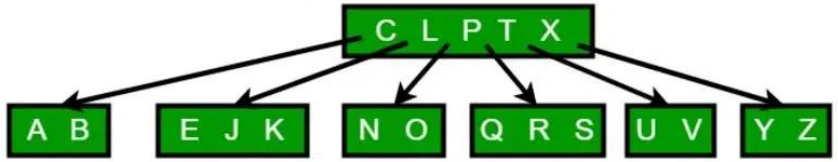


B-Tree의 삭제 과정

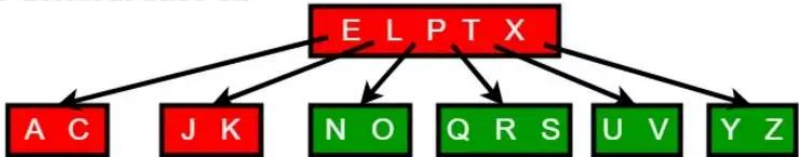
(e) D deleted: case 3b



(e') tree shrinks in height



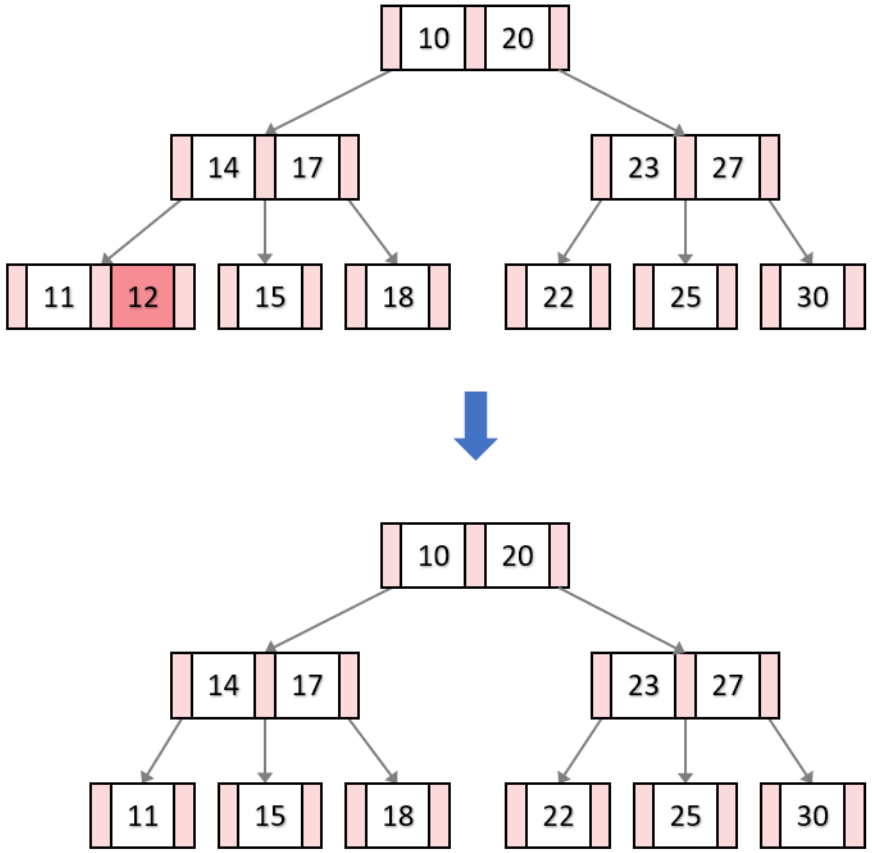
(f) B deleted: case 3a



삭제할 키가 있는 노드 검색, 실제로 키 삭제,
트리 균형 조정(필요한 경우)의 순서로 진행됨

B-Tree의 삭제 과정

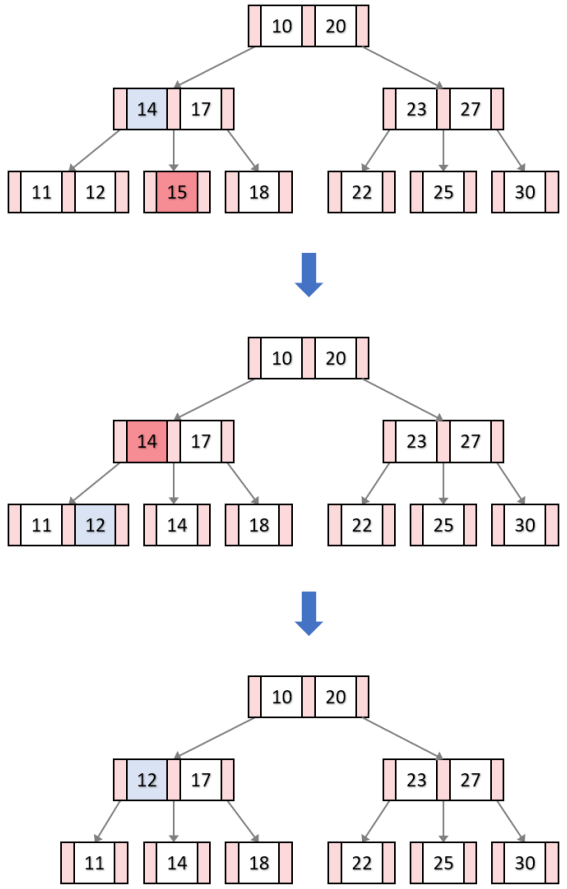
Case 1.1) 12 삭제, 해당 노드의 key 개수는 최소 key의 개수보다 크므로 바로 삭제



1 - 1. 삭제할 키가 리프에 있음 && 최소 키 개수보다 큼

B-Tree의 삭제 과정

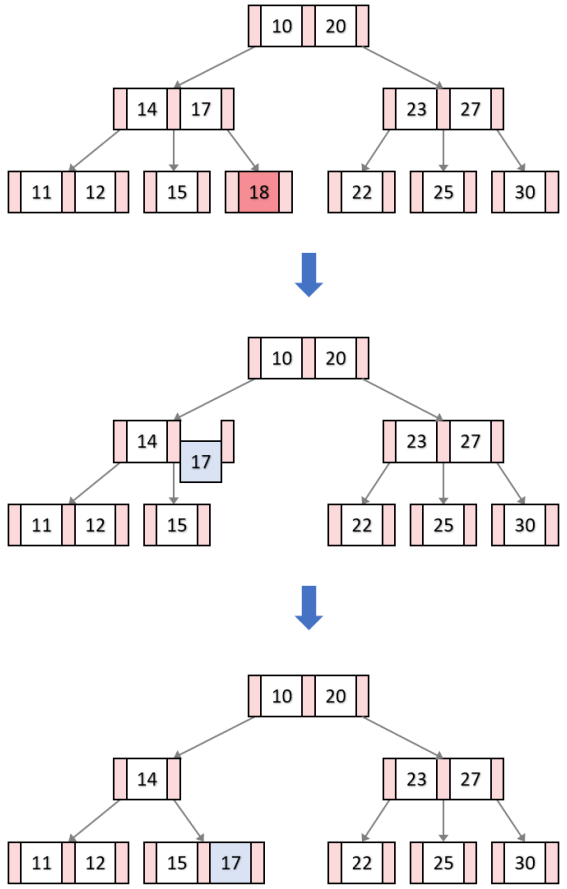
Case 1.2) 15 삭제, 해당 노드를 부모 key로 대체, 왼쪽 형제 노드에서 가장 큰 값을 부모key로 대체



1 - 2. 삭제할 키가 리프에 있음 && 형제 노드의 키가 최소 키 개수 이상

B-Tree의 삭제 과정

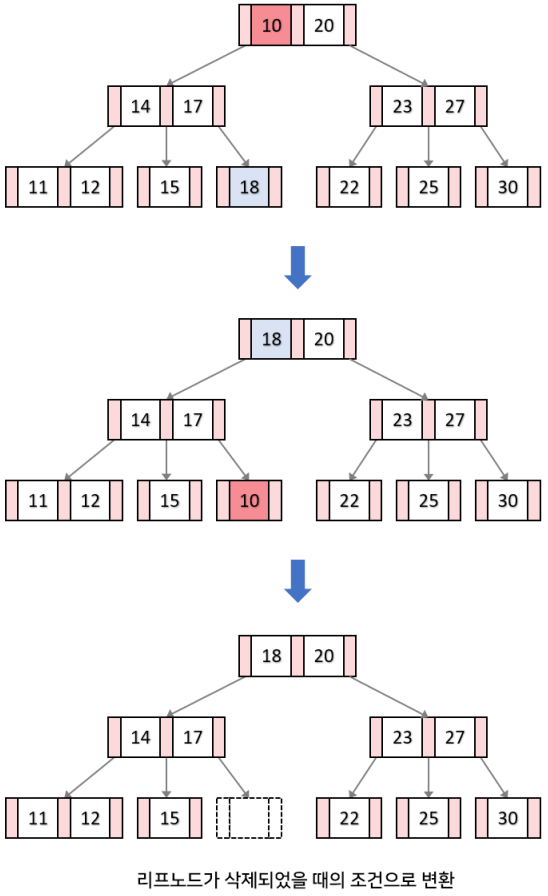
Case 1.3) 18 삭제, 부모 key를 내려 형제 노드에 병합



1 - 3. 삭제할 키가 리프에 있음 && 형제 노드의 키가 최소 키이고,
부모의 키가 최소 개수 이상

B-Tree의 삭제 과정

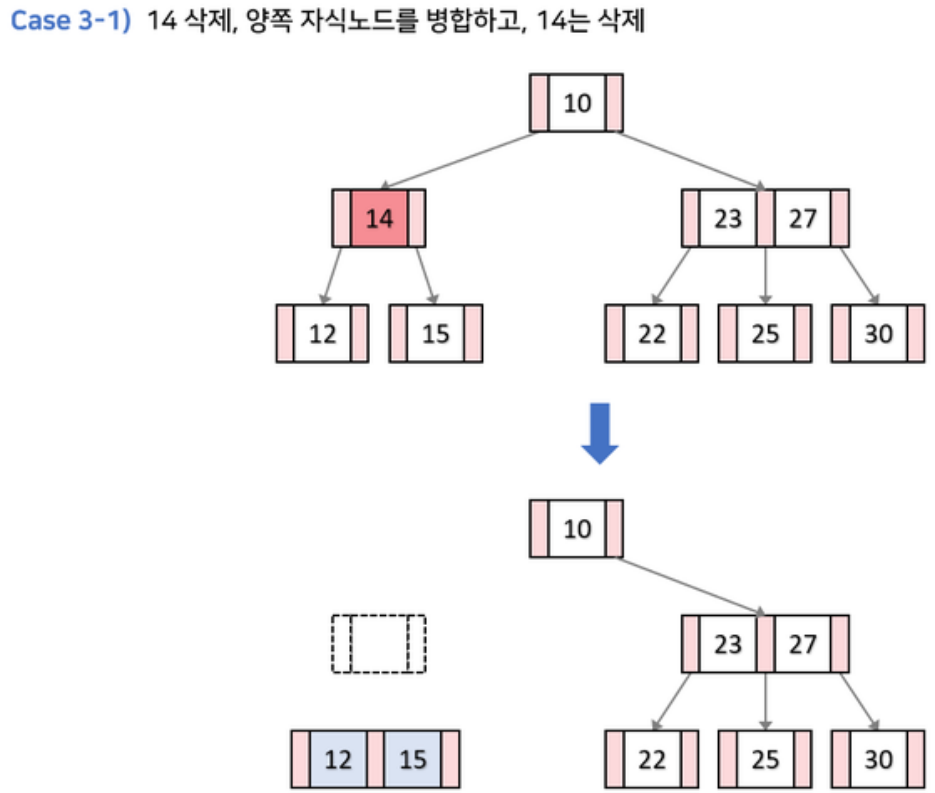
Case 2) 10 삭제, inorder predecessor인 18을 찾아 서로의 key를 교환,



리프노드가 삭제되었을 때의 조건으로 변환

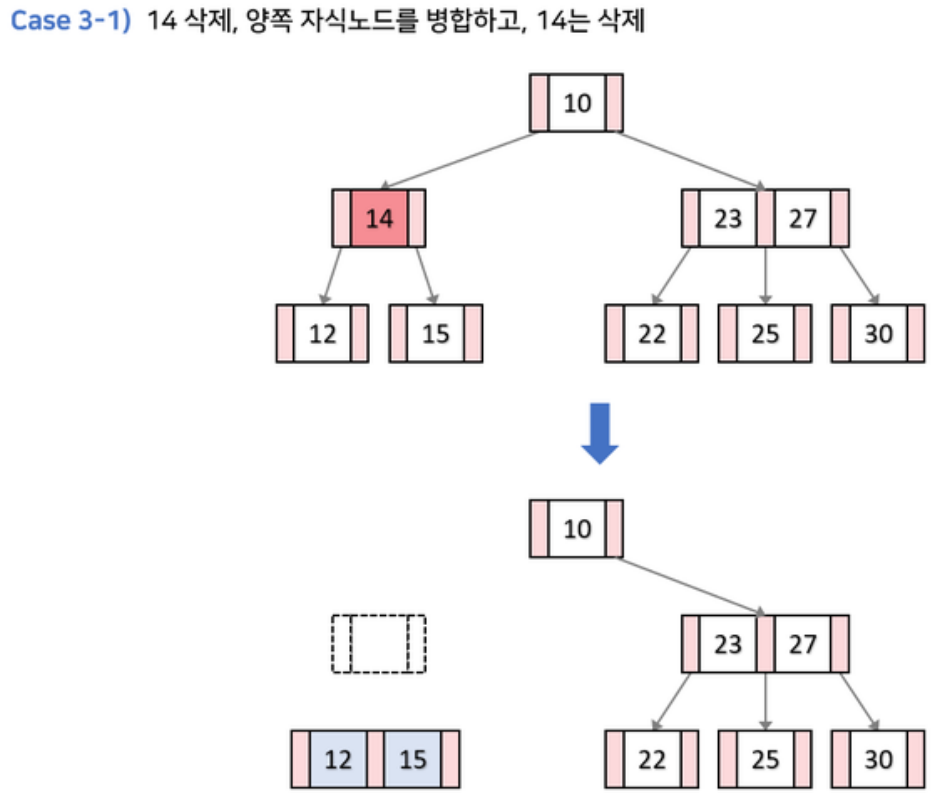
2. 삭제할 키가 리프노드를 제외한 노드에 있고,
해당 노드 또는 자식의 키가 최소 키보다 많을 경우

B-Tree의 삭제 과정



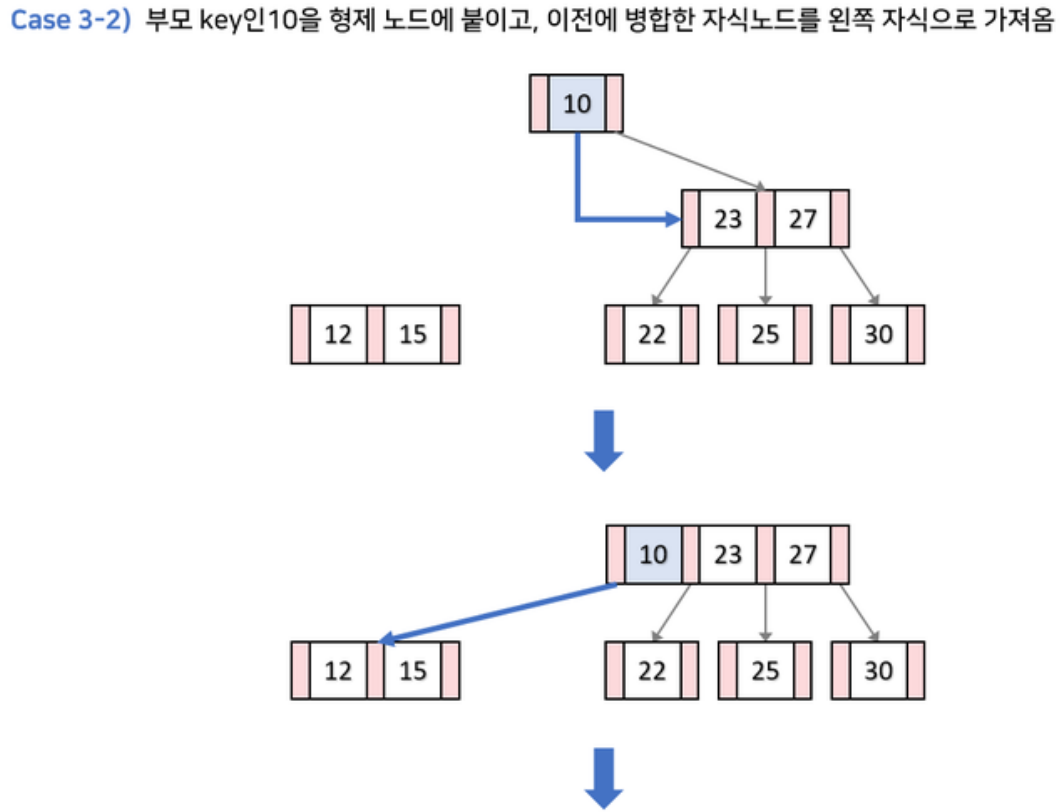
3. 삭제할 키가 내부 노드에 있고, 해당 노드와 자식 노드들의 키가 최소 키 개수만큼 있을 경우

B-Tree의 삭제 과정



3 - 1. 키를 삭제하고, 삭제한 노드의 양쪽 자식을 병합하여 하나의 노드로 만듬

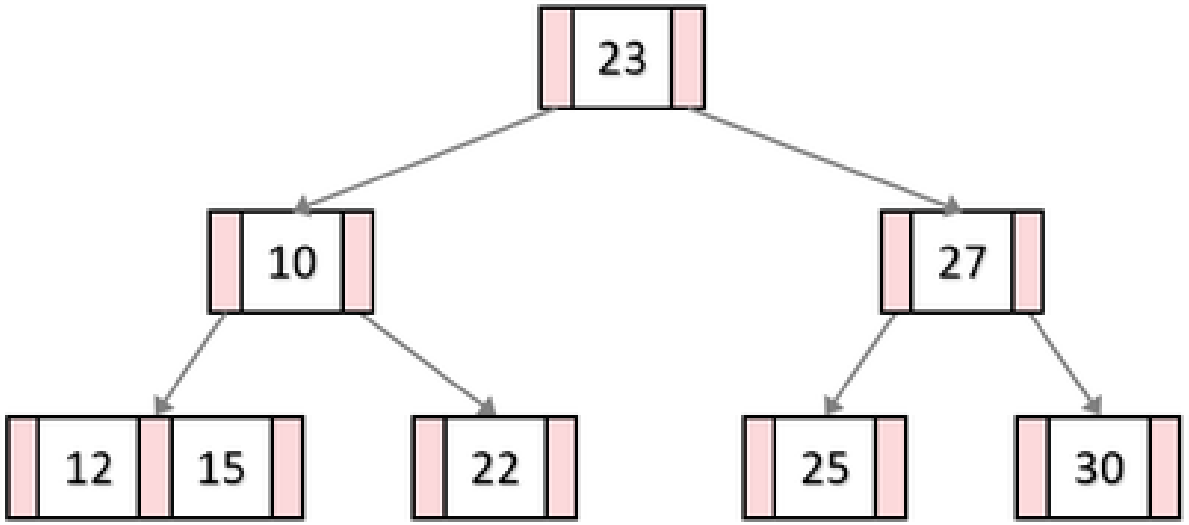
B-Tree의 삭제 과정



3 - 2. 삭제한 노드의 부모 키를 형제 노드에 붙이고,
이전에 병합한 자식 노드를 왼쪽 자식으로 가져옴

B-Tree의 삭제 과정

Case 3-3-1) 최대 key수를 넘어간 노드가 생겼기 때문에 노드분할 수행



3 - 3. 최대 키 수를 넘어간 노드가 생겼으므로 노드를 분할함

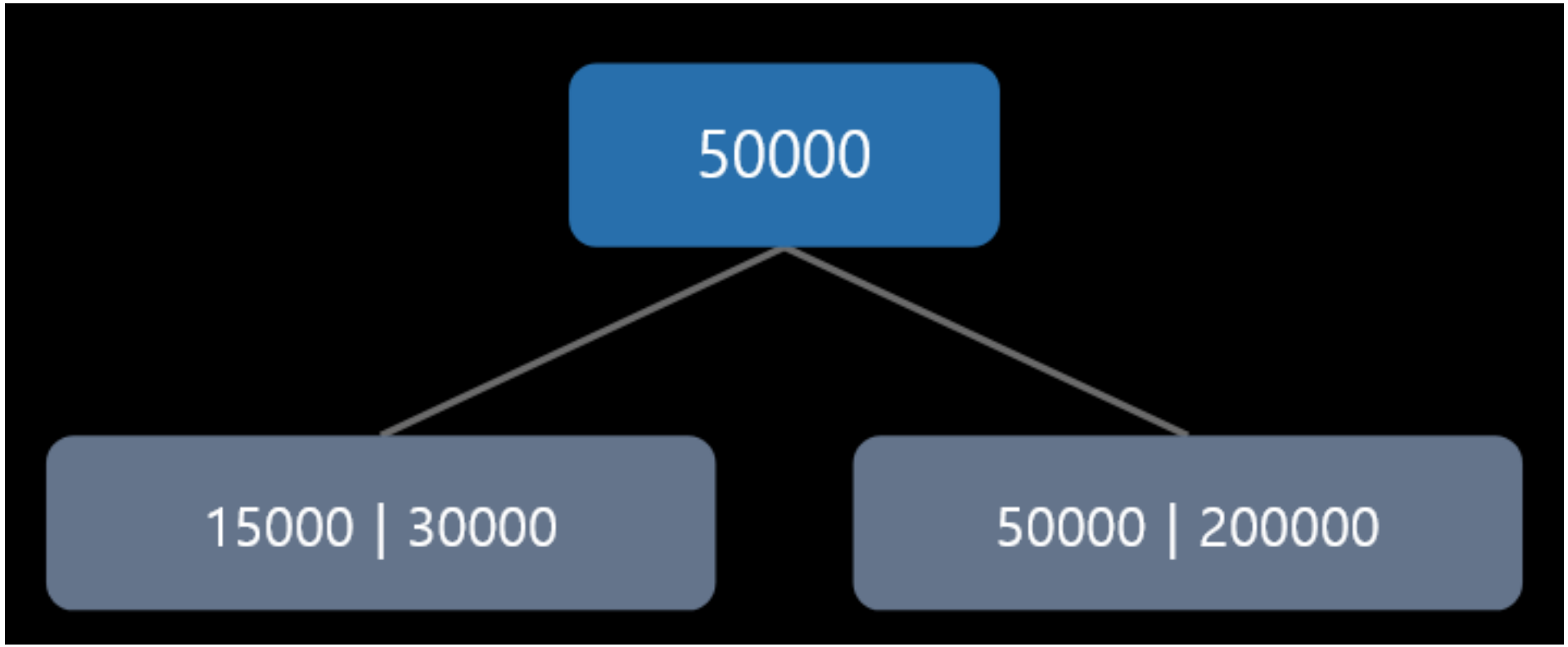
B-Tree를 이용한 DB 인덱싱

product_id	name	price
101	Keyboard	30000
102	Mouse	15000
103	Monitor	200000
104	Webcam	50000

```
CREATE INDEX idx_price
ON products(price);
```

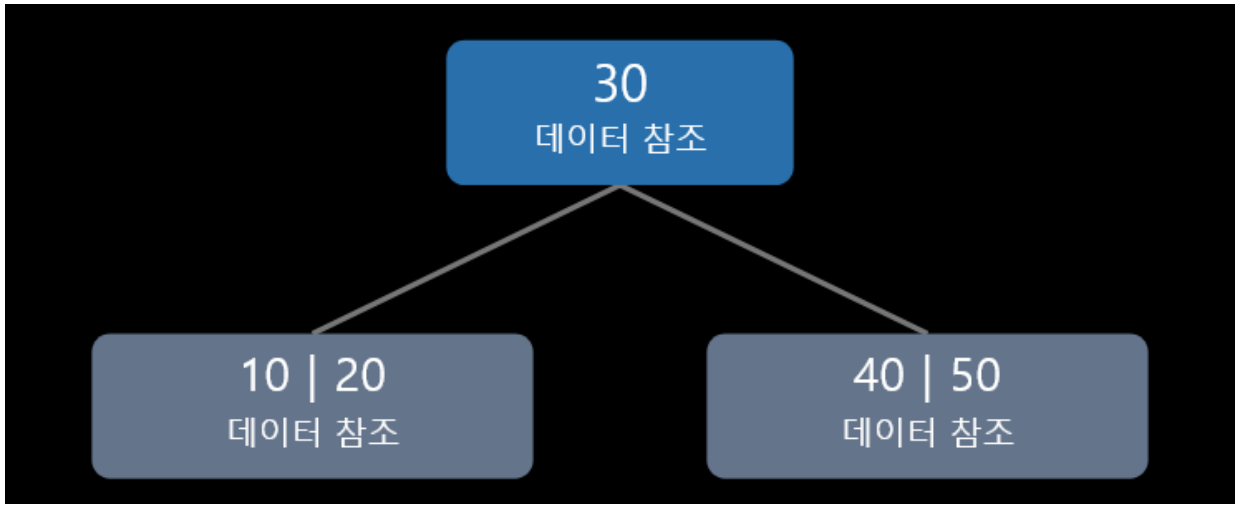
위 SQL을 실행하면 DBMS는 price를 Key로 하는 인덱스 생성

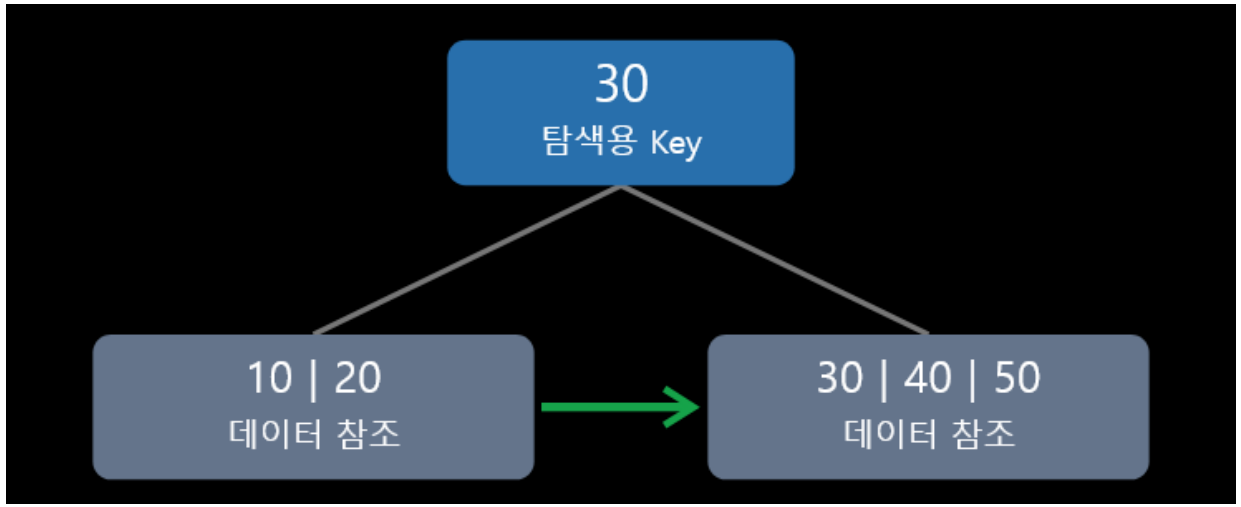
B-Tree를 이용한 DB 인덱싱



생성된 B-Tree의 예시
이 B-Tree를 순회하며 쿼리의 값을 검색함

B+Tree



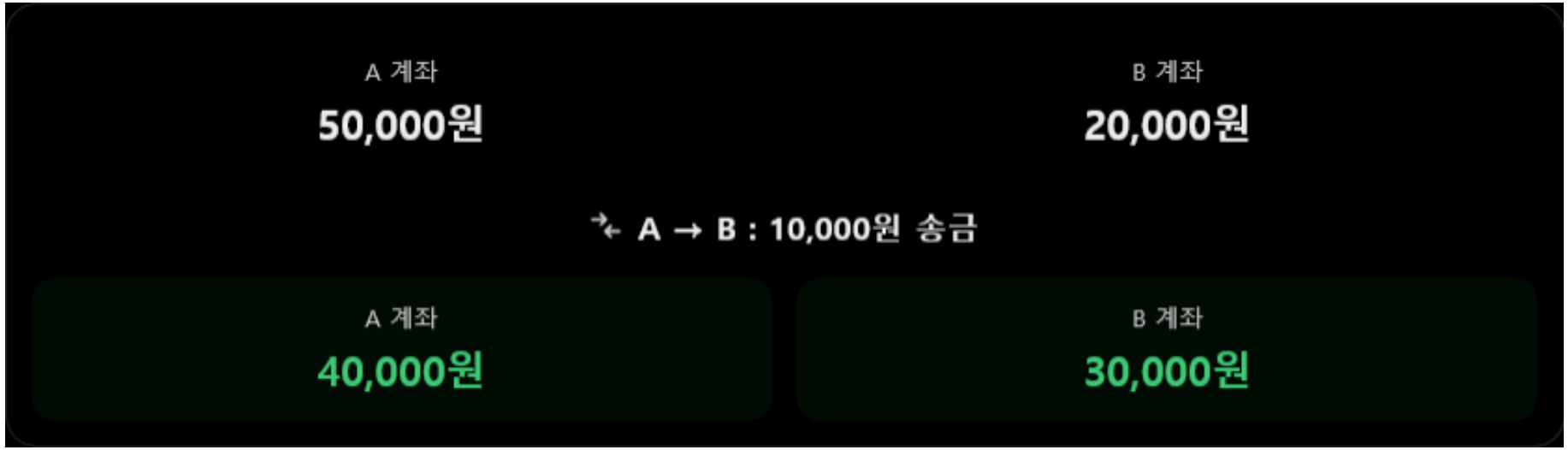


B-Tree의 리프 노드들을 서로 연결해주면 B+Tree가 됨

04
DB 트랜잭션

DB Transaction

DB Transaction



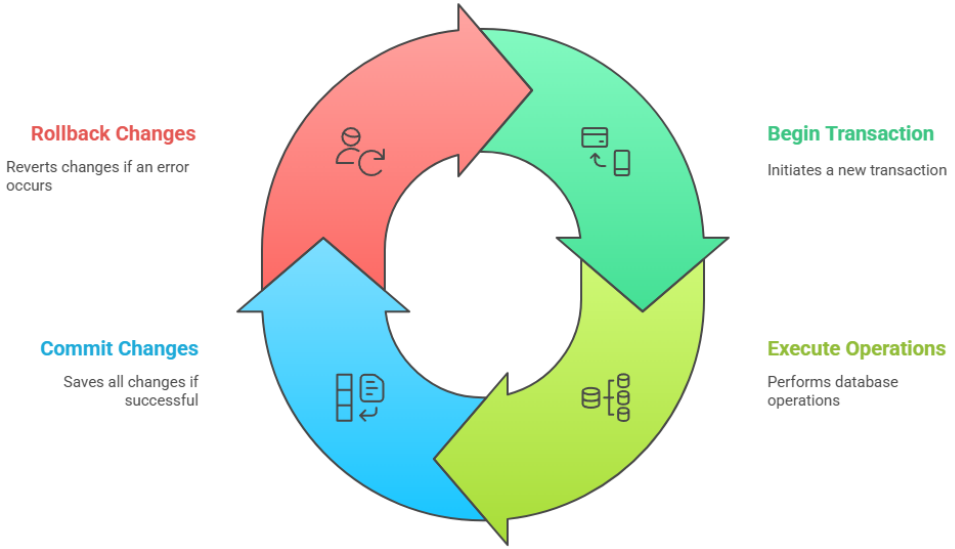

```sql
-- A 계좌에서 10,000원 차감
UPDATE accounts
SET balance = balance - 10000
WHERE id = 'A';


-- B 계좌에 10,000원 추가
UPDATE accounts
SET balance = balance + 10000
WHERE id = 'B';
```

차감 SQL은 성공했는데 추가 SQL에서 오류가 발생한다면?

→ 두 SQL을 하나로 묶기

DB Transaction




```sql
BEGIN TRANSACTION;

-- A 계좌에서 10,000원 차감
UPDATE accounts
SET balance = balance - 10000
WHERE id = 'A';

-- B 계좌에 10,000원 추가
UPDATE accounts
SET balance = balance + 10000
WHERE id = 'B';

COMMIT;
```

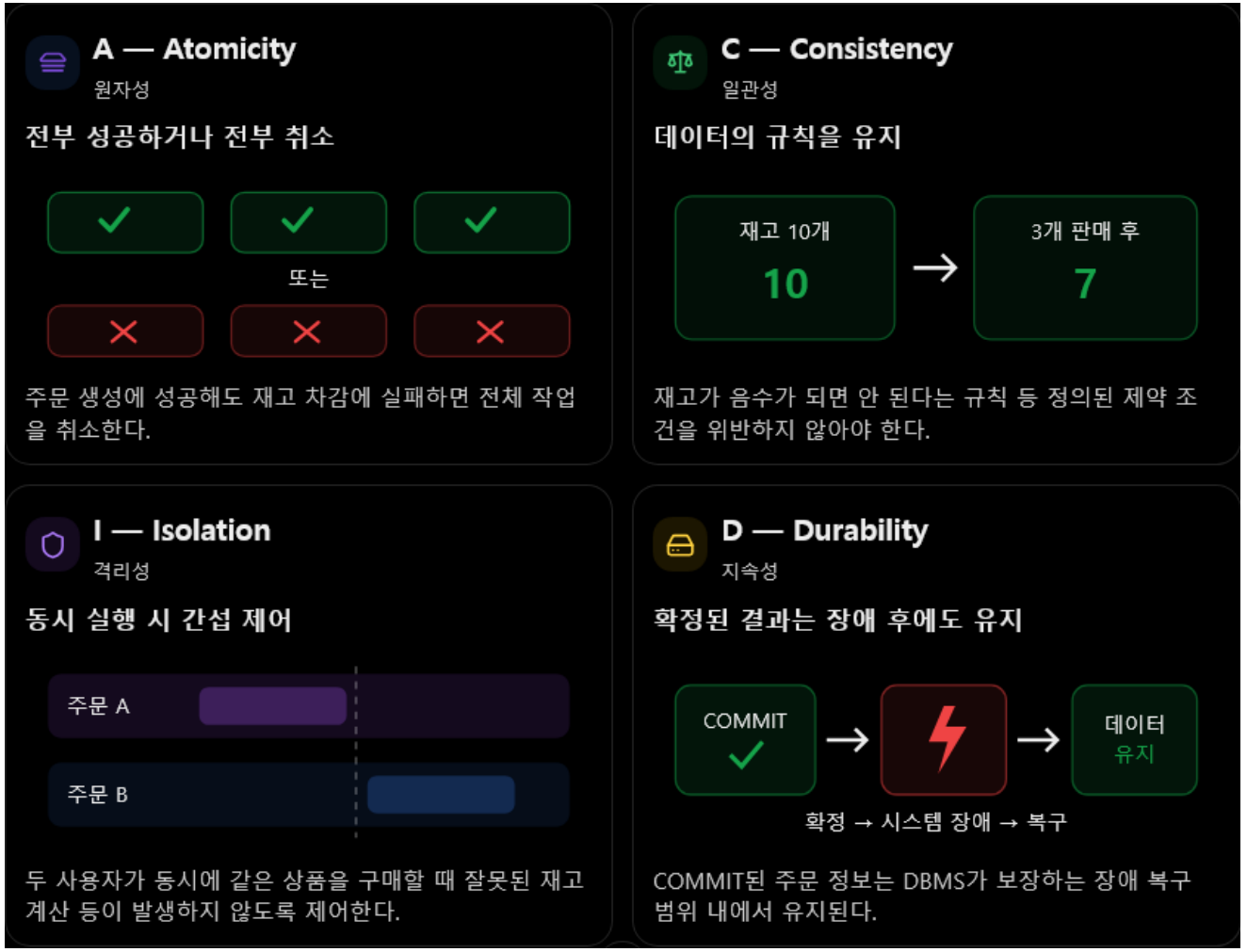
여러 명령어를 하나의 블록으로 묶어 한꺼번에 실행하고,

중간에 오류가 발생하면 ROLLBACK으로 변경사항을 취소하며,

모든 작업이 성공하면 COMMIT으로 변경 사항 확정

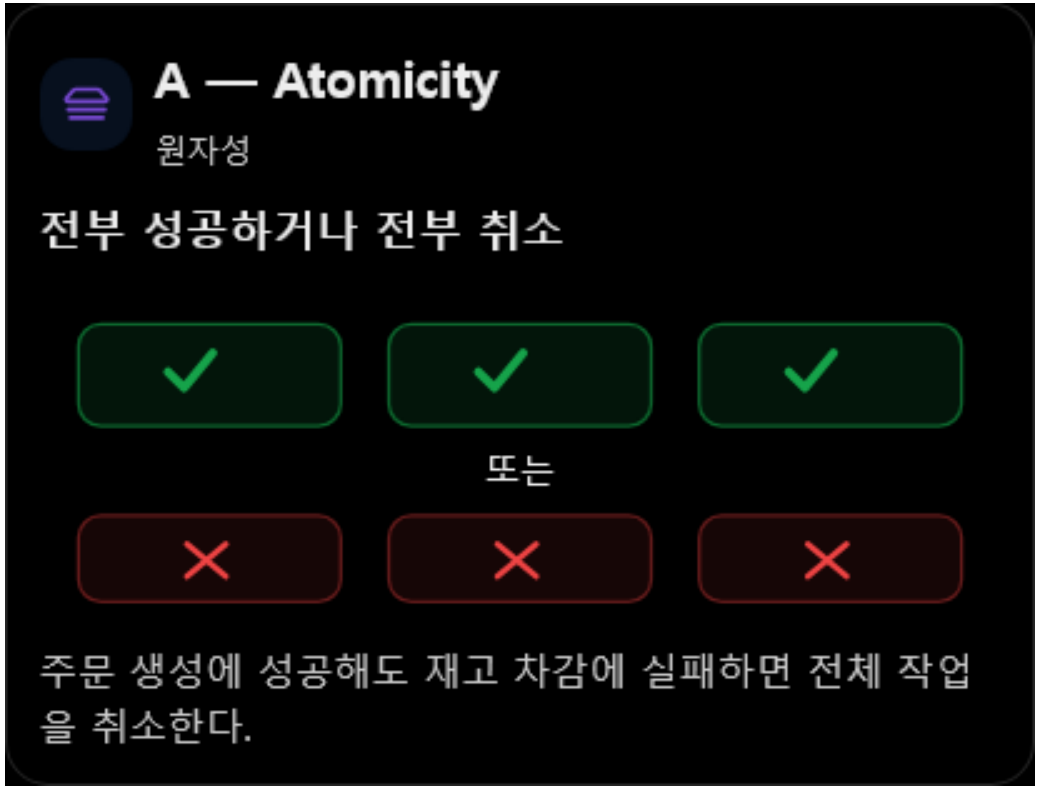
04 트랜잭션 Transaction

ACID



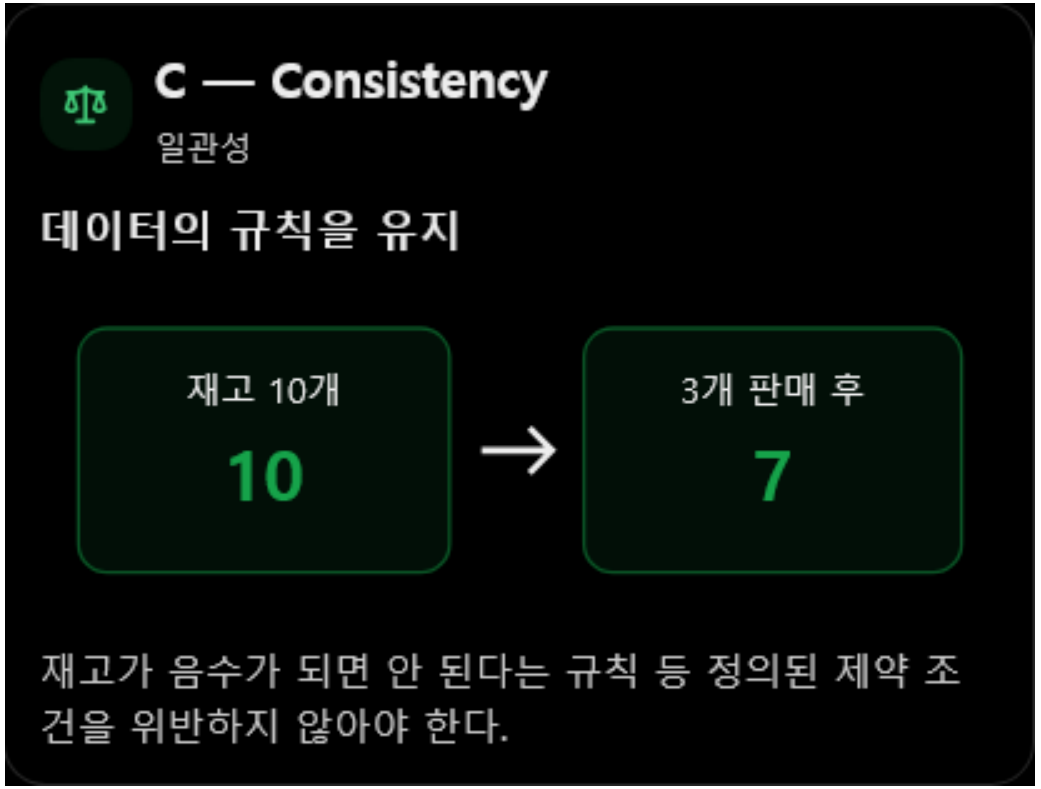
트랜잭션이 데이터를 안전하게 다루기 위해 갖추어야 하는 성질

Atomicity



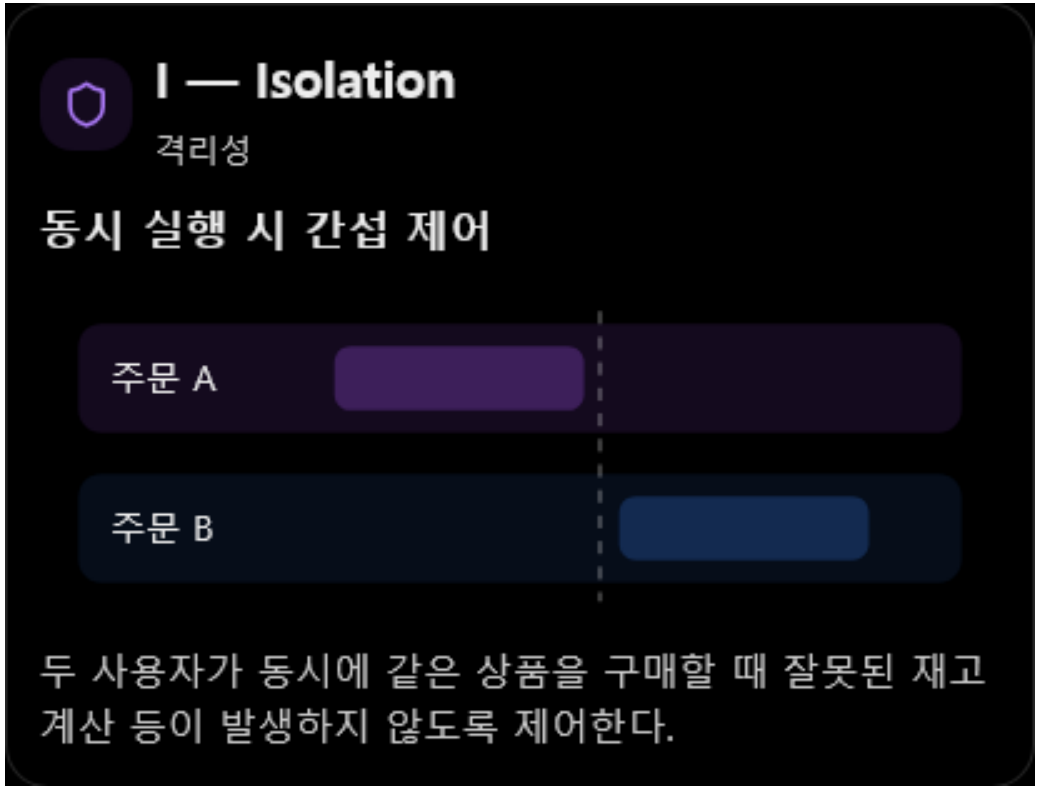
모든 작업이 성공해야만 진행, 하나라도 실패하면 전체가 실패

Consistency



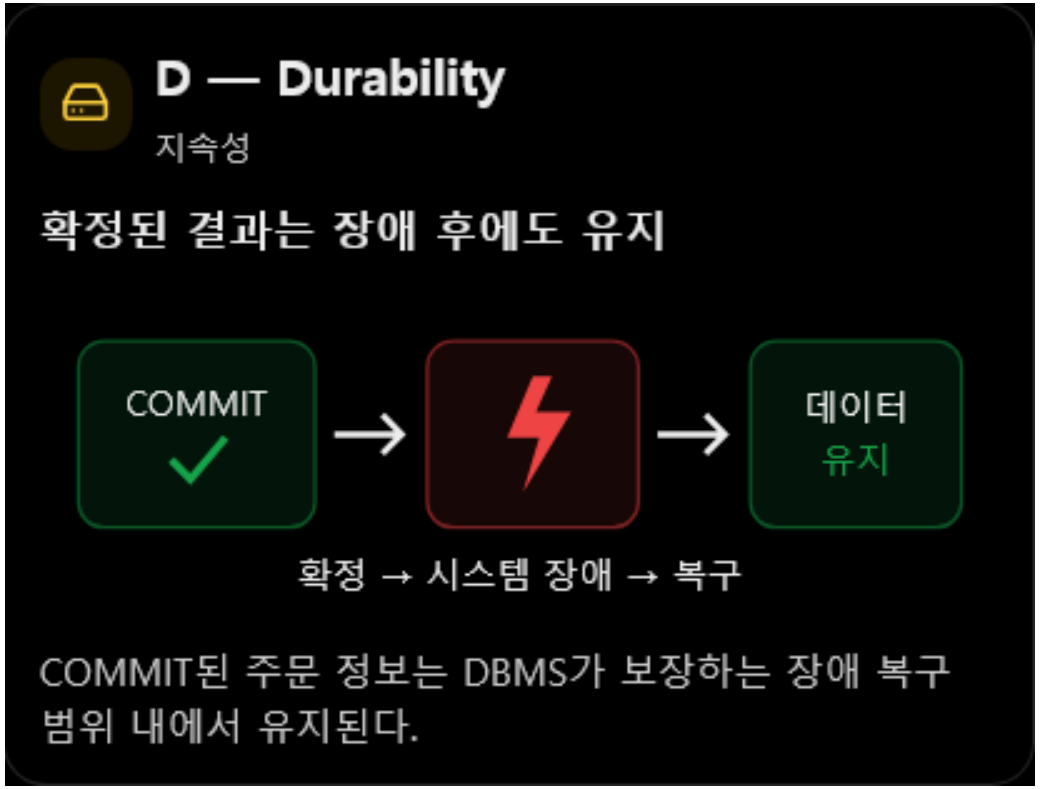
데이터는 정해진 규칙을 항상 준수해야 함

Isolation



동시에 실행되는 여러 트랜잭션이 서로 간섭하면 안됨

Durability



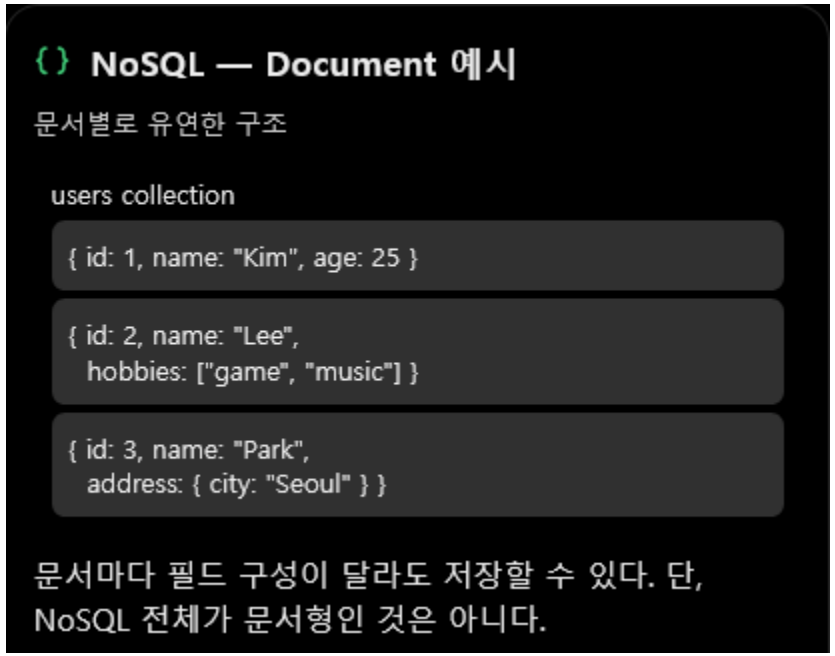
시스템 장애가 발생해도 이전 커밋은 사라지면 안됨

Relational DB



여러 테이블로 구성되며, 테이블 사이의 관계로 데이터를 모델링함

NoSQL: Not Only SQL



테이블 중심의 구조가 아닌 다른 여러 형태로

데이터를 모델링하는 데이터베이스들의 통칭

NoSQL



테이블을 사용하지 않고, 데이터에 따라

필드(모델링 방식)를 유연히 변경할 수 있음

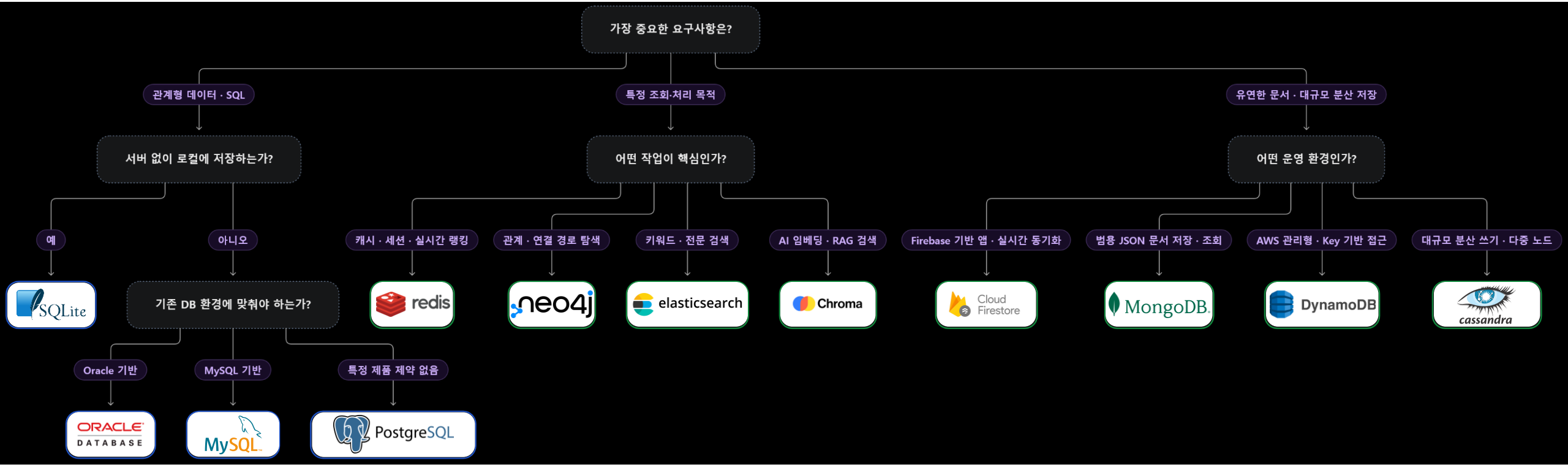
Redis



redis

- 대표적인 Key-Value 기반 데이터베이스
  - 주로 메모리에서 데이터 처리
- 빠른 조회가 필요한 캐시 또는 세션 관리에 주로 사용됨

DB 선택 가이드


가장 중요한 요구사항은?
관계형 데이터 · SQL
특정 조회·처리 목적
유연한 문서 · 대규모 분산 저장
서버 없이 로컬에 저장하는가?
어떤 작업이 핵심인가?
어떤 운영 환경인가?
예
아니오
캐시 · 세션 · 실시간 랭킹
관계 · 연결 경로 탐색
키워드 · 전문 검색
AI 임베딩 · RAG 검색
Firebase 기반 앱 · 실시간 동기화
범용 JSON 문서 저장 · 조회
AWS 관리형 · Key 기반 접근
대규모 분산 쓰기 · 다중 노드
SQLite
기존 DB 환경에 맞춰야 하는가?
redis
neo4j
elasticsearch
Chroma
Cloud Firestore
MongoDB.
DynamoDB
cassandra
Oracle 기반
MySQL 기반
특정 제품 제약 없음
ORACLE DATABASE
MySQL
PostgreSQL